

SQL

INTERVIEW MASTERY

100 Scenario-Based Questions & Answers

For 0–2 Years Experience

10 Topics

100 Questions

All with Tables

Step-by-Step SQL

Topics Covered

01	SELECT & Filtering
02	Aggregate Functions & GROUP BY
03	JOINS (INNER, LEFT, RIGHT, FULL, SELF, CROSS)
04	Subqueries (IN, EXISTS, Correlated, ALL, ANY)
05	Window Functions (ROW_NUMBER, RANK, LAG, LEAD)
06	String Functions
07	Date & Time Functions
08	NULL Handling & CASE WHEN

09	CTEs & Recursive Queries
10	DDL, DML, Indexes & Transactions

Each question includes: Scenario • Input Table • SQL Query • Output Table • Explanation • Key Tip

CHAPTER 01

SELECT & Filtering

Master WHERE, LIKE, IN, BETWEEN, DISTINCT, ORDER BY, and NULL checks.

Q1.1

Retrieve Employees from a Specific Department

Scenario: Your manager asks: 'Show me all employees working in the IT department with their salary and city.'

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
emp_id,
name,
dept,
city,
salary
FROM employees
WHERE dept = 'IT';
```

Output / Result Set:

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
3	Charlie	IT	Hyderabad	90000
6	Fatima	IT	Chennai	82000

Explanation: The WHERE clause filters rows where dept equals 'IT'. String values must be in single quotes. Only rows matching the condition are returned.

★ **Key Point:** String comparisons in SQL are case-sensitive in most databases. Use UPPER(dept) = 'IT' to be safe.

Q1.2

Find Employees with Salary Between Two Values

Scenario: HR needs a list of employees earning between ₹45,000 and ₹80,000 to plan a performance bonus.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
emp_id,
name,
dept,
salary
FROM employees
WHERE salary BETWEEN 45000 AND 80000
ORDER BY salary DESC;
```

Output / Result Set:

emp_id	name	dept	salary
1	Alice	IT	75000
4	Diana	Finance	60000
7	George	Finance	55000
8	Hannah	Marketing	48000

emp_id	name	dept	salary
2	Bob	HR	45000

Explanation: BETWEEN is inclusive — it includes both the lower (45000) and upper (80000) boundary values. ORDER BY salary DESC sorts from highest to lowest salary.

★ **Key Point:** BETWEEN 45000 AND 80000 is equivalent to salary >= 45000 AND salary <= 80000.

Q1.3

Find Employees Whose Name Starts with a Letter

Scenario: Find all employees whose name starts with the letter 'A' or 'B' for an alphabetical directory.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
emp_id,
name,
dept
FROM employees
WHERE name LIKE 'A%'
OR name LIKE 'B%'
ORDER BY name;
```

Output / Result Set:

emp_id	name	dept
1	Alice	IT
2	Bob	HR

Explanation: LIKE with wildcard '%' matches any sequence of characters. 'A%' means names starting with A. OR allows multiple LIKE conditions.

★ **Key Point:** % = any number of characters, _ = exactly one character. Example: 'A_i%' matches 'Ali', 'Alix', etc.

Q1.4

Find All Distinct Departments

Scenario: You need a clean list of all unique departments in the company for a dropdown menu in an HR portal.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT DISTINCT
dept
FROM employees
ORDER BY dept;
```

Output / Result Set:

dept
Finance
HR
IT
Marketing

Explanation: DISTINCT removes duplicate department values. Without DISTINCT, IT would appear 3 times, HR twice, etc.

★ **Key Point:** DISTINCT works on the combination of all selected columns. SELECT DISTINCT dept, city returns unique (dept, city) pairs.

Q1.5

Find Employees from Multiple Specific Cities

Scenario: The office admin needs to contact employees located in Bangalore or Chennai for a local event.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
emp_id,
name,
dept,
city
FROM employees
WHERE city IN ('Bangalore', 'Chennai')
ORDER BY city, name;
```

Output / Result Set:

emp_id	name	dept	city
1	Alice	IT	Bangalore
5	Ethan	HR	Bangalore
6	Fatima	IT	Chennai

Explanation: IN operator is a cleaner alternative to multiple OR conditions. WHERE city IN ('Bangalore','Chennai') is equivalent to WHERE city = 'Bangalore' OR city = 'Chennai'.

★ **Key Point:** IN is efficient for up to 10–15 values. For larger lists, use a temporary table and JOIN.

Q1.6

Find Employees with Missing Phone Numbers

Scenario: HR wants to identify employees who haven't provided their phone number so they can follow up.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
emp_id,
name,
dept,
phone
FROM employees
WHERE phone IS NULL;
```

Output / Result Set:

emp_id	name	dept	phone
3	Charlie	IT	NULL
5	Ethan	HR	NULL

Explanation: NULL represents a missing or unknown value. You CANNOT use WHERE phone = NULL — this never returns results. Always use IS NULL or IS NOT NULL.

★ **Key Point: Common mistake: WHERE phone = NULL returns 0 rows. Only IS NULL works correctly.**

Q1.7

Retrieve Top 3 Highest Paid Employees

Scenario: Finance wants the top 3 highest-paid employees for a reward program shortlist.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
-- SQL Server / MySQL 8+ / PostgreSQL
SELECT TOP 3
emp_id,
name,
dept,
salary
FROM employees
ORDER BY salary DESC;

-- MySQL alternative
-- SELECT emp_id, name, dept, salary
-- FROM employees
-- ORDER BY salary DESC LIMIT 3;
```

Output / Result Set:

emp_id	name	dept	salary
3	Charlie	IT	90000
6	Fatima	IT	82000
1	Alice	IT	75000

Explanation: TOP 3 (SQL Server) or LIMIT 3 (MySQL) returns only the first 3 rows after ordering. ORDER BY salary DESC ensures highest salary comes first.

★ **Key Point:** Always use ORDER BY with TOP/LIMIT, otherwise the result set is unpredictable.

Q1.8

Filter Employees NOT in Specific Departments

Scenario: List all employees who do NOT work in IT or Finance for a cross-training initiative.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
emp_id,
name,
dept,
salary
FROM employees
WHERE dept NOT IN ('IT', 'Finance')
ORDER BY dept, name;
```

Output / Result Set:

emp_id	name	dept	salary
2	Bob	HR	45000
5	Ethan	HR	42000
8	Hannah	Marketing	48000

Explanation: NOT IN excludes rows matching the listed values. Be careful: NOT IN with a NULL value in the list returns NO rows — a common trap.

★ Key Point: If the list might contain NULLs, use: **WHERE dept IS NOT NULL AND dept NOT IN ('IT','Finance').**

Q1.9

Multi-Column Sorting

Scenario: HR wants employees sorted by department (A-Z), and within each department, sorted by salary (highest first).

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
emp_id,
name,
dept,
salary
FROM employees
ORDER BY
dept ASC,
salary DESC;
```

Output / Result Set:

emp_id	name	dept	salary
4	Diana	Finance	60000
7	George	Finance	55000
2	Bob	HR	45000
5	Ethan	HR	42000
3	Charlie	IT	90000
6	Fatima	IT	82000
1	Alice	IT	75000
8	Hannah	Marketing	48000

Explanation: ORDER BY can sort by multiple columns. First it sorts by dept ASC (alphabetically), then within the same dept it sorts by salary DESC (highest to lowest).

★ **Key Point:** Column order in ORDER BY matters. The first column is the primary sort; subsequent columns break ties.

Q1.10

Combine Multiple Filters with AND / OR

Scenario: Find IT or Finance employees earning more than ₹55,000, hired after 2021.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
emp_id,
name,
dept,
salary,
hire_date
FROM employees
WHERE
(dept = 'IT' OR dept = 'Finance')
AND salary > 55000
AND hire_date > '2021-12-31'
ORDER BY salary DESC;
```

Output / Result Set:

emp_id	name	dept	salary	hire_date
1	Alice	IT	75000	2022-03-15

emp_id	name	dept	salary	hire_date
4	Diana	Finance	60000	2023-05-20

Explanation: Parentheses group OR conditions so they are evaluated before AND. Without parentheses, AND has higher precedence than OR, which can change results.

★ **Key Point:** Always use parentheses when mixing AND and OR to make logic explicit and avoid bugs.

CHAPTER 02

Aggregate Functions & GROUP BY

COUNT, SUM, AVG, MIN, MAX with GROUP BY, HAVING, and conditional aggregation.

Q2.1

Count Employees Per Department

Scenario: HR wants to know how many employees are in each department for headcount reporting.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
dept,
COUNT(*) AS total_employees
FROM employees
GROUP BY dept
ORDER BY total_employees DESC;
```

Output / Result Set:

dept	total_employees
IT	3
HR	2
Finance	2
Marketing	1

Explanation: COUNT(*) counts all rows including NULLs. GROUP BY dept groups rows by department value, then COUNT(*) counts rows within each group.

★ **Key Point: COUNT(*) vs COUNT(column): COUNT(*) counts all rows; COUNT(phone) skips rows where phone IS NULL.**

Q2.2

Calculate Average Salary Per Department

Scenario: Payroll team wants the average salary for each department to prepare the salary budget.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
dept,
COUNT(*) AS headcount,
AVG(salary) AS avg_salary,
MIN(salary) AS min_salary,
MAX(salary) AS max_salary
FROM employees
GROUP BY dept
ORDER BY avg_salary DESC;
```

Output / Result Set:

dept	headcount	avg_salary	min_salary	max_salary
IT	3	82333	75000	90000
Finance	2	57500	55000	60000
Marketing	1	48000	48000	48000

dept	headcount	avg_salary	min_salary	max_salary
HR	2	43500	42000	45000

Explanation: Multiple aggregate functions (AVG, MIN, MAX, COUNT) can be used in a single GROUP BY query. Each function independently calculates within each group.

★ **Key Point:** `ROUND(AVG(salary), 2)` to format decimal values neatly.

Q2.3

Find the Department with the Highest Total Salary

Scenario: Find which department has the highest total salary spend.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT TOP 1
dept,
SUM(salary) AS total_salary_spend
FROM employees
GROUP BY dept
ORDER BY total_salary_spend DESC;
```

Output / Result Set:

dept	total_salary_spend
IT	247000

Explanation: `SUM(salary)` totals the salary for each department group. `ORDER BY DESC` puts the highest-spending department first. `TOP 1` retrieves only that row.

★ **Key Point:** Remove `TOP 1` to see all departments ranked — useful for budget comparisons.

Q2.4

Filter Groups Using HAVING Clause

Scenario: Management wants departments that have more than 2 employees to consider for team restructuring.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
dept,
COUNT(*) AS employee_count
FROM employees
GROUP BY dept
HAVING COUNT(*) > 2
ORDER BY employee_count DESC;
```

Output / Result Set:

dept	employee_count
IT	3

Explanation: HAVING filters groups AFTER aggregation, while WHERE filters individual rows BEFORE aggregation. You cannot use aggregate functions (COUNT, SUM) inside WHERE.

★ **Key Point:** WHERE filters rows, HAVING filters groups. WHERE runs before GROUP BY, HAVING runs after.

Q2.5

Find Departments Whose Average Salary Exceeds ₹50,000

Scenario: Identify high-paying departments where the average salary is above ₹50,000.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
dept,
ROUND(AVG(salary), 0) AS avg_salary
FROM employees
GROUP BY dept
HAVING AVG(salary) > 50000
ORDER BY avg_salary DESC;
```

Output / Result Set:

dept	avg_salary
IT	82333
Finance	57500

Explanation: HAVING AVG(salary) > 50000 filters groups where the calculated average exceeds the threshold. ROUND() makes the output cleaner.

★ **Key Point:** You can use HAVING with SUM, AVG, COUNT, MIN, MAX — any aggregate function.

Q2.6

Total Sales Amount Per Product Category

Scenario: Sales manager needs total revenue by product category to identify the best-performing category.

Table: orders

order_id	emp_id	product	category	amount	order_date	city
101	1	Laptop	Electronics	85000	2024-01-10	Bangalore
102	2	Chair	Furniture	12000	2024-01-15	Mumbai
103	1	Mouse	Electronics	1500	2024-01-20	Bangalore
104	3	Desk	Furniture	18000	2024-02-05	Hyderabad
105	4	Monitor	Electronics	22000	2024-02-10	Delhi
106	2	Keyboard	Electronics	3500	2024-02-18	Mumbai
107	5	Notebook	Stationery	200	2024-03-01	Bangalore
108	3	Laptop	Electronics	85000	2024-03-15	Hyderabad
109	6	Phone	Electronics	45000	2024-03-20	Chennai
110	7	Pen Stand	Stationery	500	2024-03-25	Mumbai

SQL Query:

```
SELECT
category,
COUNT(order_id) AS total_orders,
SUM(amount) AS total_revenue,
ROUND(AVG(amount),0) AS avg_order_value
FROM orders
GROUP BY category
ORDER BY total_revenue DESC;
```

Output / Result Set:

category	total_orders	total_revenue	avg_order_value
Electronics	6	242000	40333
Furniture	2	30000	15000
Stationery	2	700	350

Explanation: GROUP BY category segments all orders by their category. Multiple aggregates give a comprehensive sales summary in one query.

★ **Key Point:** This pattern (GROUP BY + multiple aggregates) is one of the most common in real business queries.

Q2.7

Monthly Order Count for the Year

Scenario: Business analyst wants to see how many orders were placed each month in 2024.

Table: orders

order_id	emp_id	product	category	amount	order_date	city
101	1	Laptop	Electronics	85000	2024-01-10	Bangalore
102	2	Chair	Furniture	12000	2024-01-15	Mumbai

order_id	emp_id	product	category	amount	order_date	city
103	1	Mouse	Electronics	1500	2024-01-20	Bangalore
104	3	Desk	Furniture	18000	2024-02-05	Hyderabad
105	4	Monitor	Electronics	22000	2024-02-10	Delhi
106	2	Keyboard	Electronics	3500	2024-02-18	Mumbai
107	5	Notebook	Stationery	200	2024-03-01	Bangalore
108	3	Laptop	Electronics	85000	2024-03-15	Hyderabad
109	6	Phone	Electronics	45000	2024-03-20	Chennai
110	7	Pen Stand	Stationery	500	2024-03-25	Mumbai

SQL Query:

```
SELECT
MONTH(order_date) AS order_month,
DATENAME(MONTH, order_date) AS month_name,
COUNT(order_id) AS total_orders,
SUM(amount) AS total_revenue
FROM orders
WHERE YEAR(order_date) = 2024
GROUP BY
MONTH(order_date),
DATENAME(MONTH, order_date)
ORDER BY order_month;
```

Output / Result Set:

order_month	month_name	total_orders	total_revenue
1	January	3	98500
2	February	3	43500
3	March	4	130700

Explanation: MONTH() and DATENAME() extract parts from dates. GROUP BY must include all non-aggregated columns in the SELECT list.

★ **Key Point:** In MySQL use MONTHNAME(order_date) instead of DATENAME(MONTH, order_date).

Q2.8

City-Wise Employee Count and Total Salary

Scenario: Office admin needs a city-wise summary of employee headcount and total salary for cost allocation.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
city,
COUNT(*) AS employees,
SUM(salary) AS total_salary
FROM employees
GROUP BY city
ORDER BY total_salary DESC;
```

Output / Result Set:

city	employees	total_salary
Bangalore	2	117000
Mumbai	2	100000
Hyderabad	1	90000
Chennai	1	82000
Delhi	2	108000

Explanation: City-wise grouping aggregates all employees in the same city. Useful for location-based cost reports and resource planning.

★ **Key Point:** Add **HAVING COUNT(*) > 1** to show only cities with more than one employee.

Q2.9

Find Products Ordered More Than Once

Scenario: Inventory team wants to identify which products have been ordered more than once to plan restocking.

Table: orders

order_id	emp_id	product	category	amount	order_date	city
101	1	Laptop	Electronics	85000	2024-01-10	Bangalore
102	2	Chair	Furniture	12000	2024-01-15	Mumbai
103	1	Mouse	Electronics	1500	2024-01-20	Bangalore
104	3	Desk	Furniture	18000	2024-02-05	Hyderabad
105	4	Monitor	Electronics	22000	2024-02-10	Delhi
106	2	Keyboard	Electronics	3500	2024-02-18	Mumbai
107	5	Notebook	Stationery	200	2024-03-01	Bangalore
108	3	Laptop	Electronics	85000	2024-03-15	Hyderabad
109	6	Phone	Electronics	45000	2024-03-20	Chennai
110	7	Pen Stand	Stationery	500	2024-03-25	Mumbai

SQL Query:

```
SELECT
product,
category,
COUNT(order_id) AS times_ordered,
SUM(amount) AS total_amount
FROM orders
GROUP BY product, category
HAVING COUNT(order_id) > 1
ORDER BY times_ordered DESC;
```

Output / Result Set:

product	category	times_ordered	total_amount
Laptop	Electronics	2	170000

Explanation: GROUP BY on multiple columns (product, category) creates groups for each unique product-category combination. HAVING COUNT > 1 keeps only repeated items.

★ **Key Point:** GROUP BY with multiple columns: each unique combination of those columns forms one group.

Q2.10

Conditional Count Using CASE WHEN Inside COUNT

Scenario: HR needs to count how many employees earn above and below ₹60,000 in each department.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210

emp_id	name	dept	city	salary	hire_date	phone
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
dept,
COUNT(*) AS total,
COUNT(CASE WHEN salary > 60000 THEN 1 END)
AS high_earners,
COUNT(CASE WHEN salary <= 60000 THEN 1 END)
AS low_earners
FROM employees
GROUP BY dept
ORDER BY dept;
```

Output / Result Set:

dept	total	high_earners	low_earners
Finance	2	0	2
HR	2	0	2
IT	3	3	0
Marketing	1	0	1

Explanation: CASE WHEN inside COUNT is called Conditional Aggregation. COUNT only counts non-NULL values — when the CASE condition is false, it returns NULL, which is not counted.

★ **Key Point:** This is a powerful pattern to pivot data without a PIVOT function. Also works with SUM(CASE WHEN...).

CHAPTER 03

JOINS

INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER, SELF JOIN, CROSS JOIN, and multi-table joins.

Q3.1

INNER JOIN – Employees with Their Department Names

Scenario: Show each employee's name and the name of their department (only employees who have a department).

Table: employees

emp_id	name	dept_id	salary	manager_id
1	Alice	10	75000	5
2	Bob	20	45000	4
3	Charlie	10	90000	NULL
4	Diana	30	60000	3
5	Ethan	20	42000	3
6	Fatima	10	82000	3
7	George	NULL	55000	NULL

Table: departments

dept_id	dept_name	location
10	IT	Bangalore
20	HR	Mumbai
30	Finance	Delhi
40	Marketing	Chennai

SQL Query:

```
SELECT
  e.emp_id,
  e.name AS employee_name,
  d.dept_name,
  d.location
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id
ORDER BY e.emp_id;
```

Output / Result Set:

emp_id	employee_name	dept_name	location
1	Alice	IT	Bangalore
2	Bob	HR	Mumbai
3	Charlie	IT	Bangalore
4	Diana	Finance	Delhi
5	Ethan	HR	Mumbai
6	Fatima	IT	Bangalore

Explanation: INNER JOIN returns only rows where the join condition matches in BOTH tables. George (dept_id=NULL) and Marketing (no employees) are both excluded.

★ **Key Point:** INNER JOIN is the default JOIN type. Writing just JOIN also means INNER JOIN.

Q3.2

LEFT JOIN – All Employees Including Unassigned

Scenario: Show ALL employees even if they have no department assigned.

Table: employees

emp_id	name	dept_id	salary	manager_id
1	Alice	10	75000	5
2	Bob	20	45000	4
3	Charlie	10	90000	NULL
4	Diana	30	60000	3
5	Ethan	20	42000	3
6	Fatima	10	82000	3
7	George	NULL	55000	NULL

Table: departments

dept_id	dept_name	location
10	IT	Bangalore
20	HR	Mumbai
30	Finance	Delhi
40	Marketing	Chennai

SQL Query:

```
SELECT
  e.emp_id,
  e.name AS employee_name,
  d.dept_name,
  d.location
FROM employees e
```

```

LEFT JOIN departments d
ON e.dept_id = d.dept_id
ORDER BY e.emp_id;

```

Output / Result Set:

emp_id	employee_name	dept_name	location
1	Alice	IT	Bangalore
2	Bob	HR	Mumbai
3	Charlie	IT	Bangalore
4	Diana	Finance	Delhi
5	Ethan	HR	Mumbai
6	Fatima	IT	Bangalore
7	George	NULL	NULL

Explanation: LEFT JOIN returns ALL rows from the LEFT table (employees), and matching rows from the RIGHT table. When there's no match, NULL fills the right side.

★ **Key Point:** Use LEFT JOIN when you want to keep all rows from one table regardless of matches.

Q3.3

Find Departments with No Employees

Scenario: Management wants to know which departments currently have no employees assigned.

Table: employees

emp_id	name	dept_id	salary	manager_id
1	Alice	10	75000	5
2	Bob	20	45000	4
3	Charlie	10	90000	NULL
4	Diana	30	60000	3
5	Ethan	20	42000	3
6	Fatima	10	82000	3
7	George	NULL	55000	NULL

Table: departments

dept_id	dept_name	location
10	IT	Bangalore
20	HR	Mumbai
30	Finance	Delhi
40	Marketing	Chennai

SQL Query:

```

SELECT
d.dept_id,
d.dept_name,
d.location
FROM departments d
LEFT JOIN employees e
ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL
ORDER BY d.dept_id;

```

Output / Result Set:

dept_id	dept_name	location
40	Marketing	Chennai

Explanation: Reverse the LEFT JOIN (put departments on the left). WHERE e.emp_id IS NULL filters to rows with no employee match — i.e., empty departments.

★ **Key Point:** LEFT JOIN + WHERE right_table.id IS NULL = Anti-join pattern. Great for finding unmatched records.

Q3.4

FULL OUTER JOIN – All Records from Both Tables

Scenario: Show all employees AND all departments, matching where possible, NULLs where not.

Table: employees

emp_id	name	dept_id	salary	manager_id
1	Alice	10	75000	5
2	Bob	20	45000	4
3	Charlie	10	90000	NULL
4	Diana	30	60000	3
5	Ethan	20	42000	3
6	Fatima	10	82000	3
7	George	NULL	55000	NULL

Table: departments

dept_id	dept_name	location
10	IT	Bangalore
20	HR	Mumbai
30	Finance	Delhi
40	Marketing	Chennai

SQL Query:

```

SELECT

```

```

e.emp_id,
e.name AS employee_name,
d.dept_id,
d.dept_name
FROM employees e
FULL OUTER JOIN departments d
ON e.dept_id = d.dept_id
ORDER BY e.emp_id;

```

Output / Result Set:

emp_id	employee_name	dept_id	dept_name
1	Alice	10	IT
2	Bob	20	HR
3	Charlie	10	IT
4	Diana	30	Finance
5	Ethan	20	HR
6	Fatima	10	IT
7	George	NULL	NULL
NULL	NULL	40	Marketing

Explanation: FULL OUTER JOIN returns ALL rows from BOTH tables. Unmatched rows get NULLs for the other table's columns. Combines LEFT JOIN and RIGHT JOIN results.

★ **Key Point:** MySQL doesn't support FULL OUTER JOIN directly. Use: LEFT JOIN UNION RIGHT JOIN.

Q3.5

SELF JOIN – Find Employee and Their Manager

Scenario: Show each employee's name alongside their manager's name from the same employees table.

Table: employees

emp_id	name	dept_id	salary	manager_id
1	Alice	10	75000	5
2	Bob	20	45000	4
3	Charlie	10	90000	NULL
4	Diana	30	60000	3
5	Ethan	20	42000	3
6	Fatima	10	82000	3
7	George	NULL	55000	NULL

SQL Query:

```
SELECT
```

```

e.emp_id,
e.name AS employee_name,
m.name AS manager_name
FROM employees e
LEFT JOIN employees m
ON e.manager_id = m.emp_id
ORDER BY e.emp_id;

```

Output / Result Set:

emp_id	employee_name	manager_name
1	Alice	Ethan
2	Bob	Diana
3	Charlie	NULL
4	Diana	Charlie
5	Ethan	Charlie
6	Fatima	Charlie
7	George	NULL

Explanation: SELF JOIN joins a table to itself using two aliases (e = employee, m = manager). LEFT JOIN keeps employees even if they have no manager (like Charlie and George).

★ **Key Point:** SELF JOINS are used for hierarchical/recursive data stored in the same table (org charts, categories).

Q3.6

Three-Table JOIN

Scenario: Show employee names, their department names, and department location — joining 3 tables.

Table: employees

emp_id	name	dept_id	salary	manager_id
1	Alice	10	75000	5
2	Bob	20	45000	4
3	Charlie	10	90000	NULL
4	Diana	30	60000	3
5	Ethan	20	42000	3
6	Fatima	10	82000	3
7	George	NULL	55000	NULL

Table: departments

dept_id	dept_name	location
10	IT	Bangalore

dept_id	dept_name	location
20	HR	Mumbai
30	Finance	Delhi
40	Marketing	Chennai

Table: projects

proj_id	emp_id	project_name	status
101	1	Website Revamp	Active
102	3	AI Module	Active
103	6	Cloud Migration	Done
104	2	HR Portal	Active

SQL Query:

```
SELECT
e.name AS employee,
d.dept_name,
p.project_name,
p.status
FROM employees e
INNER JOIN departments d
ON e.dept_id = d.dept_id
INNER JOIN projects p
ON e.emp_id = p.emp_id
ORDER BY e.name;
```

Output / Result Set:

employee	dept_name	project_name	status
Alice	IT	Website Revamp	Active
Bob	HR	HR Portal	Active
Charlie	IT	AI Module	Active
Fatima	IT	Cloud Migration	Done

Explanation: Chain INNER JOINS to connect multiple tables. Each JOIN adds columns from one more table based on a relationship key.

★ **Key Point:** Always alias tables (e, d, p) when joining multiple tables to avoid ambiguous column names.

Q3.7

CROSS JOIN – Generate All Possible Combinations

Scenario: Create all possible shirt combinations of colors and sizes for a product catalog.

Table: colors

color
Red
Blue
Black

Table: sizes

size
S
M
L

SQL Query:

```
SELECT
c.color,
s.size,
CONCAT(c.color, ' - ', s.size)
AS product_variant
FROM colors c
CROSS JOIN sizes s
ORDER BY c.color, s.size;
```

Output / Result Set:

color	size	product_variant
Black	L	Black - L
Black	M	Black - M
Black	S	Black - S
Blue	L	Blue - L
Blue	M	Blue - M
Blue	S	Blue - S
Red	L	Red - L
Red	M	Red - M
Red	S	Red - S

Explanation: CROSS JOIN produces a Cartesian product — every row from Table A combined with every row from Table B. 3 colors x 3 sizes = 9 combinations.

★ **Key Point:** CROSS JOIN has no ON clause. Use with caution on large tables — 1000 rows x 1000 rows = 1,000,000 rows.

Q3.8

JOIN with Aggregation – Department Headcount

Scenario: Show each department name and how many employees work in it (using JOIN + GROUP BY).

Table: employees

emp_id	name	dept_id	salary	manager_id
1	Alice	10	75000	5
2	Bob	20	45000	4
3	Charlie	10	90000	NULL
4	Diana	30	60000	3
5	Ethan	20	42000	3
6	Fatima	10	82000	3
7	George	NULL	55000	NULL

Table: departments

dept_id	dept_name	location
10	IT	Bangalore
20	HR	Mumbai
30	Finance	Delhi
40	Marketing	Chennai

SQL Query:

```
SELECT
d.dept_name,
d.location,
COUNT(e.emp_id) AS employee_count,
SUM(e.salary) AS total_salary
FROM departments d
LEFT JOIN employees e
ON d.dept_id = e.dept_id
GROUP BY d.dept_name, d.location
ORDER BY employee_count DESC;
```

Output / Result Set:

dept_name	location	employee_count	total_salary
IT	Bangalore	3	247000
HR	Mumbai	2	87000
Finance	Delhi	1	60000
Marketing	Chennai	0	NULL

Explanation: Combining LEFT JOIN with GROUP BY allows aggregating data across related tables. COUNT(e.emp_id) returns 0 for Marketing because it counts non-NULL emp_ids.

★ **Key Point:** Use COUNT(e.emp_id) not COUNT(*) with LEFT JOIN — COUNT(*) would count the NULL row as 1.

Q3.9

JOIN to Find Employees Without Any Project

Scenario: HR wants employees who have not been assigned to any project.

Table: employees

emp_id	name	dept_id	salary	manager_id
1	Alice	10	75000	5
2	Bob	20	45000	4
3	Charlie	10	90000	NULL
4	Diana	30	60000	3
5	Ethan	20	42000	3
6	Fatima	10	82000	3
7	George	NULL	55000	NULL

Table: projects

proj_id	emp_id	project_name
101	1	Website
102	3	AI Module
103	6	Cloud
104	2	HR Portal

SQL Query:

```
SELECT
e.emp_id,
e.name,
e.salary
FROM employees e
LEFT JOIN projects p
ON e.emp_id = p.emp_id
WHERE p.proj_id IS NULL
ORDER BY e.emp_id;
```

Output / Result Set:

emp_id	name	salary
4	Diana	60000
5	Ethan	42000
7	George	55000

Explanation: LEFT JOIN + WHERE right_key IS NULL is the anti-join pattern. Rows that don't match in the right table get NULL for all right columns.

★ **Key Point:** This pattern replaces NOT IN (subquery) and is often faster on large tables.

Q3.10

JOIN on Multiple Conditions

Scenario: Find employee-manager pairs where both are in the same department.

Table: employees

emp_id	name	dept_id	salary	manager_id
1	Alice	10	75000	5
2	Bob	20	45000	4
3	Charlie	10	90000	NULL
4	Diana	30	60000	3
5	Ethan	20	42000	3
6	Fatima	10	82000	3
7	George	NULL	55000	NULL

SQL Query:

```
SELECT
e.name AS employee,
e.dept_id AS emp_dept,
m.name AS manager,
m.dept_id AS mgr_dept
FROM employees e
INNER JOIN employees m
ON e.manager_id = m.emp_id
AND e.dept_id = m.dept_id
ORDER BY e.name;
```

Output / Result Set:

employee	emp_dept	manager	mgr_dept
Alice	10	Fatima	10
Fatima	10	Charlie	10
Ethan	20	Bob	20

Explanation: JOIN ON can have multiple conditions using AND. This filters to only pairs where both the employee-manager relationship exists AND they share the same dept_id.

★ **Key Point:** Multi-condition JOINS are cleaner than WHERE clause filters — the JOIN condition documents the relationship intent.

CHAPTER 04

Subqueries

Scalar, multi-row, correlated subqueries, IN, NOT IN, EXISTS, NOT EXISTS, ALL, ANY.

Q4.1

Find Employees Earning Above Average Salary

Scenario: Show employees whose salary is higher than the company-wide average salary.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
emp_id,
name,
dept,
salary
FROM employees
WHERE salary > (
SELECT AVG(salary)
FROM employees
)
ORDER BY salary DESC;
```

Output / Result Set:

emp_id	name	dept	salary
3	Charlie	IT	90000

emp_id	name	dept	salary
6	Fatima	IT	82000
1	Alice	IT	75000
4	Diana	Finance	60000

Explanation: The inner query (subquery) runs first and returns the average salary (59625). The outer query then uses that value in the WHERE condition.

★ **Key Point:** The subquery runs once and returns a single value (scalar subquery). This is different from a correlated subquery.

Q4.2

Find the Second Highest Salary

Scenario: Classic interview question: find the employee with the second highest salary.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
-- Method 1: Subquery
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (
  SELECT MAX(salary) FROM employees
);

-- Method 2: With employee details
SELECT TOP 1
emp_id, name, salary
FROM employees
WHERE salary < (
```

```
SELECT MAX(salary) FROM employees
)
ORDER BY salary DESC;
```

Output / Result Set:

emp_id	name	salary
6	Fatima	82000

Explanation: First, the inner query finds MAX salary (90000). The outer query then finds the MAX salary that is strictly less than 90000, which gives the 2nd highest.

★ **Key Point: For Nth highest salary: ORDER BY salary DESC OFFSET N-1 ROWS FETCH NEXT 1 ROW ONLY (SQL Server 2012+).**

Q4.3

Subquery with IN – Employees in High-Earning Departments

Scenario: Find employees who belong to departments where the average salary exceeds ₹60,000.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
emp_id,
name,
dept,
salary
FROM employees
WHERE dept IN (
SELECT dept
```

```

FROM employees
GROUP BY dept
HAVING AVG(salary) > 60000
)
ORDER BY dept, salary DESC;

```

Output / Result Set:

emp_id	name	dept	salary
3	Charlie	IT	90000
6	Fatima	IT	82000
1	Alice	IT	75000

Explanation: The inner subquery returns a list of departments (IT) where avg salary > 60000. The outer query uses IN to filter employees from those departments.

★ **Key Point:** Subquery with IN can return multiple rows — this is a multi-row subquery, unlike scalar subqueries.

Q4.4

Correlated Subquery – Employees Earning Above Dept Average

Scenario: Find employees who earn more than the average salary of their own department.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```

SELECT
e1.emp_id,
e1.name,
e1.dept,

```

```

e1.salary,
(
SELECT ROUND(AVG(e2.salary),0)
FROM employees e2
WHERE e2.dept = e1.dept
) AS dept_avg_salary
FROM employees e1
WHERE e1.salary > (
SELECT AVG(e2.salary)
FROM employees e2
WHERE e2.dept = e1.dept
)
ORDER BY e1.dept, e1.salary DESC;

```

Output / Result Set:

emp_id	name	dept	salary	dept_avg_salary
3	Charlie	IT	90000	82333
6	Fatima	IT	82000	82333
4	Diana	Finance	60000	57500
2	Bob	HR	45000	43500
8	Hannah	Marketing	48000	48000

Explanation: A correlated subquery references the outer query (e1.dept). It runs ONCE PER ROW of the outer query, using the current row's dept to compute that department's avg salary.

★ **Key Point:** Correlated subqueries are powerful but slow on large tables. Use a CTE or JOIN + window function for better performance.

Q4.5

EXISTS – Departments That Have Employees

Scenario: List departments that have at least one employee assigned.

Table: employees

emp_id	name	dept_id	salary	manager_id
1	Alice	10	75000	5
2	Bob	20	45000	4
3	Charlie	10	90000	NULL
4	Diana	30	60000	3
5	Ethan	20	42000	3
6	Fatima	10	82000	3
7	George	NULL	55000	NULL

Table: departments

dept_id	dept_name	location
10	IT	Bangalore
20	HR	Mumbai
30	Finance	Delhi
40	Marketing	Chennai

SQL Query:

```
SELECT
dept_id,
dept_name,
location
FROM departments d
WHERE EXISTS (
SELECT 1
FROM employees e
WHERE e.dept_id = d.dept_id
)
ORDER BY dept_id;
```

Output / Result Set:

dept_id	dept_name	location
10	IT	Bangalore
20	HR	Mumbai
30	Finance	Delhi

Explanation: EXISTS returns TRUE if the subquery returns at least one row. SELECT 1 is a convention — EXISTS doesn't care WHAT the subquery returns, only WHETHER it returns rows.

★ **Key Point:** EXISTS stops scanning as soon as the first matching row is found, making it faster than IN for large subquery results.

Q4.6

NOT EXISTS – Departments with No Employees

Scenario: Find departments that have no employees assigned.

Table: employees

emp_id	name	dept_id	salary	manager_id
1	Alice	10	75000	5
2	Bob	20	45000	4
3	Charlie	10	90000	NULL
4	Diana	30	60000	3

emp_id	name	dept_id	salary	manager_id
5	Ethan	20	42000	3
6	Fatima	10	82000	3
7	George	NULL	55000	NULL

Table: departments

dept_id	dept_name	location
10	IT	Bangalore
20	HR	Mumbai
30	Finance	Delhi
40	Marketing	Chennai

SQL Query:

```
SELECT
dept_id,
dept_name,
location
FROM departments d
WHERE NOT EXISTS (
SELECT 1
FROM employees e
WHERE e.dept_id = d.dept_id
);
```

Output / Result Set:

dept_id	dept_name	location
40	Marketing	Chennai

Explanation: NOT EXISTS is the opposite — it returns rows from the outer query where the subquery returns NO rows. This is the cleanest way to find 'orphan' records.

★ **Key Point:** NOT EXISTS is NULL-safe, unlike NOT IN. If a NULL exists in the subquery result, NOT IN returns no rows; NOT EXISTS handles it correctly.

Q4.7

Subquery in FROM Clause (Derived Table)

Scenario: Find departments where total salary is in the top 2.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678

emp_id	name	dept	city	salary	hire_date	phone
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```

SELECT TOP 2
dept_summary.dept,
dept_summary.total_salary,
dept_summary.avg_salary
FROM (
SELECT
dept,
SUM(salary) AS total_salary,
ROUND(AVG(salary), 0) AS avg_salary
FROM employees
GROUP BY dept
) AS dept_summary
ORDER BY dept_summary.total_salary DESC;

```

Output / Result Set:

dept	total_salary	avg_salary
IT	247000	82333
Finance	115000	57500

Explanation: A subquery in the FROM clause is called a Derived Table or Inline View. It acts like a temporary table for that query. Always give it an alias (AS dept_summary).

★ **Key Point:** Derived tables are not reusable within the same query. Use a CTE (WITH clause) when you need to reference the same subquery multiple times.

Q4.8

Subquery with ALL – Employees Earning More Than All HR Staff

Scenario: Find employees who earn more than every single HR department employee.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
emp_id,
name,
dept,
salary
FROM employees
WHERE salary > ALL (
SELECT salary
FROM employees
WHERE dept = 'HR'
)
ORDER BY salary DESC;
```

Output / Result Set:

emp_id	name	dept	salary
3	Charlie	IT	90000
6	Fatima	IT	82000
1	Alice	IT	75000
4	Diana	Finance	60000
7	George	Finance	55000
8	Hannah	Marketing	48000

Explanation: salary > ALL(subquery) means salary must be greater than EVERY value in the subquery result. HR max is 45000. Only employees earning > 45000 qualify.

★ **Key Point:** > ALL is equivalent to > MAX(subquery). < ALL is equivalent to < MIN(subquery).

Q4.9

Subquery with ANY – Find Employees with Salary Matching Any IT Salary

Scenario: Find employees whose salary equals any salary in the IT department.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
emp_id,
name,
dept,
salary
FROM employees
WHERE salary = ANY (
SELECT salary
FROM employees
WHERE dept = 'IT'
)
ORDER BY salary DESC;
```

Output / Result Set:

emp_id	name	dept	salary
3	Charlie	IT	90000
6	Fatima	IT	82000
1	Alice	IT	75000

Explanation: = ANY is equivalent to IN. It returns rows where the column matches at least one value in the subquery. In this case, only IT employees happen to have those exact salaries.

★ **Key Point:** ANY is rarely used in practice. Use IN instead — it's more readable and identical in behavior for equality checks.

Q4.10

Nested Subquery – Top Earner Per Department

Scenario: Find the employee with the highest salary in each department.

Table: employees

emp_id	name	dept	city	salary	hire_date	phone
1	Alice	IT	Bangalore	75000	2022-03-15	9876543210
2	Bob	HR	Mumbai	45000	2021-07-01	9812345678
3	Charlie	IT	Hyderabad	90000	2020-01-10	NULL
4	Diana	Finance	Delhi	60000	2023-05-20	9900112233
5	Ethan	HR	Bangalore	42000	2022-11-30	NULL
6	Fatima	IT	Chennai	82000	2021-04-18	9988776655
7	George	Finance	Mumbai	55000	2019-08-25	9911223344
8	Hannah	Marketing	Delhi	48000	2023-01-07	9876001122

SQL Query:

```
SELECT
  e.emp_id,
  e.name,
  e.dept,
  e.salary
FROM employees e
WHERE e.salary = (
  SELECT MAX(e2.salary)
  FROM employees e2
  WHERE e2.dept = e.dept
)
ORDER BY e.salary DESC;
```

Output / Result Set:

emp_id	name	dept	salary
3	Charlie	IT	90000

emp_id	name	dept	salary
4	Diana	Finance	60000
8	Hannah	Marketing	48000
2	Bob	HR	45000

Explanation: This correlated subquery finds the MAX salary within each employee's own department. The outer query then matches only employees with that max salary.

★ **Key Point: Modern SQL: use ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) for cleaner syntax.**

CHAPTER 05

Window Functions

ROW_NUMBER, RANK, DENSE_RANK, PARTITION BY, LAG, LEAD, NTILE, running totals.

Q5.1

ROW_NUMBER() – Unique Rank for Every Row

Scenario: Assign a unique sequential row number to every employee ordered by salary (highest first).

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
SELECT
emp_id,
name,
dept,
salary,
ROW_NUMBER() OVER (
ORDER BY salary DESC
) AS row_num
FROM employees
ORDER BY row_num;
```

Output / Result Set:

emp_id	name	dept	salary	row_num
3	Charlie	IT	90000	1
6	Fatima	IT	82000	2
1	Alice	IT	75000	3
4	Diana	Finance	60000	4
7	George	Finance	55000	5

emp_id	name	dept	salary	row_num
8	Hannah	Marketing	48000	6
2	Bob	HR	45000	7
5	Ethan	HR	45000	8

Explanation: ROW_NUMBER() always gives unique sequential numbers. Even though Bob and Ethan have the same salary (45000), they get different row numbers (7 and 8). The tie-breaking is arbitrary.

★ **Key Point:** ROW_NUMBER never produces ties. Use RANK or DENSE_RANK when you need to handle duplicate values.

Q5.2

RANK() vs DENSE_RANK() – Handling Tied Values

Scenario: Rank employees by salary. Show the difference between RANK and DENSE_RANK when there are ties.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
SELECT
name,
salary,
RANK() OVER (ORDER BY salary DESC)
AS rank_col,
DENSE_RANK() OVER (ORDER BY salary DESC)
AS dense_rank_col,
ROW_NUMBER() OVER (ORDER BY salary DESC)
AS row_num
FROM employees
ORDER BY salary DESC;
```

Output / Result Set:

name	salary	rank_col	dense_rank_col	row_num
Charlie	90000	1	1	1
Fatima	82000	2	2	2
Alice	75000	3	3	3
Diana	60000	4	4	4
George	55000	5	5	5
Hannah	48000	6	6	6
Bob	45000	7	7	7
Ethan	45000	7	7	8

Explanation: RANK: tied values get same rank, next rank is skipped (7,7,9).
DENSE_RANK: tied values same rank, next rank NOT skipped (7,7,8). ROW_NUMBER: always unique.

★ **Key Point: Interview favourite: RANK gives gaps after ties, DENSE_RANK does not. Remember: D for Dense = No gaps.**

Q5.3

PARTITION BY – Rank Employees Within Each Department

Scenario: Rank employees by salary within their own department (each department has its own 1,2,3 ranking).

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
SELECT
name,
dept,
salary,
DENSE_RANK() OVER (
PARTITION BY dept
ORDER BY salary DESC
) AS dept_salary_rank
```

```
FROM employees
ORDER BY dept, salary DESC;
```

Output / Result Set:

name	dept	salary	dept_salary_rank
Diana	Finance	60000	1
George	Finance	55000	2
Bob	HR	45000	1
Ethan	HR	45000	1
Charlie	IT	90000	1
Fatima	IT	82000	2
Alice	IT	75000	3
Hannah	Marketing	48000	1

Explanation: PARTITION BY dept restarts the ranking counter for each department. Window functions calculate across a 'window' of rows defined by PARTITION BY and ORDER BY.

★ **Key Point:** PARTITION BY in window functions is like GROUP BY, but it does NOT reduce rows — all original rows are preserved.

Q5.4

Top N Per Group Using ROW_NUMBER()

Scenario: Find the top 2 highest-paid employees in each department.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
SELECT
emp_id, name, dept, salary, dept_rank
FROM (
SELECT
emp_id,
name,
```

```

dept,
salary,
ROW_NUMBER() OVER (
PARTITION BY dept
ORDER BY salary DESC
) AS dept_rank
FROM employees
) ranked
WHERE dept_rank <= 2
ORDER BY dept, dept_rank;

```

Output / Result Set:

emp_id	name	dept	salary	dept_rank
4	Diana	Finance	60000	1
7	George	Finance	55000	2
2	Bob	HR	45000	1
5	Ethan	HR	45000	2
3	Charlie	IT	90000	1
6	Fatima	IT	82000	2
8	Hannah	Marketing	48000	1

Explanation: Window functions can't be used directly in WHERE. Wrap in a subquery or CTE, then filter by the window function result in the outer query.

★ **Key Point:** This Top-N per group pattern is extremely common in interviews. Know it by heart.

Q5.5

LAG() – Compare Current Row with Previous Row

Scenario: For each employee, show their salary and the salary of the employee ranked just below them (previous row's salary).

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
SELECT
name,
dept,
salary,
LAG(salary, 1, 0) OVER (
ORDER BY salary DESC
) AS prev_salary,
salary - LAG(salary,1,0) OVER (
ORDER BY salary DESC
) AS salary_diff
FROM employees
ORDER BY salary DESC;
```

Output / Result Set:

name	dept	salary	prev_salary	salary_diff
Charlie	IT	90000	0	90000
Fatima	IT	82000	90000	-8000
Alice	IT	75000	82000	-7000
Diana	Finance	60000	75000	-15000
George	Finance	55000	60000	-5000
Hannah	Marketing	48000	55000	-7000
Bob	HR	45000	48000	-3000
Ethan	HR	45000	45000	0

Explanation: LAG(column, offset, default) accesses the value N rows BEFORE the current row. For the first row, there is no previous row, so the default value (0) is used.

★ **Key Point:** LAG = look back. LEAD = look forward. LAG(salary,1) vs LAG(salary,2) for 1 or 2 rows back.

Q5.6

LEAD() – Compare Current Row with Next Row

Scenario: Show each employee's salary and the next higher salary to understand the gap they need to close.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000

emp_id	name	dept	city	salary
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
SELECT
name,
dept,
salary,
LEAD(salary, 1, 0) OVER (
ORDER BY salary DESC
) AS next_lower_salary,
salary - LEAD(salary,1,0) OVER (
ORDER BY salary DESC
) AS gap_to_next
FROM employees
ORDER BY salary DESC;
```

Output / Result Set:

name	dept	salary	next_lower_salary	gap_to_next
Charlie	IT	90000	82000	8000
Fatima	IT	82000	75000	7000
Alice	IT	75000	60000	15000
Diana	Finance	60000	55000	5000
George	Finance	55000	48000	7000
Hannah	Marketing	48000	45000	3000
Bob	HR	45000	45000	0
Ethan	HR	45000	0	45000

Explanation: LEAD(column, offset, default) looks FORWARD — it accesses the value of N rows AFTER the current row. Useful for time-series comparisons and period-over-period analysis.

★ **Key Point:** LAG and LEAD are commonly used in financial analysis (month-over-month growth) and sequential event analysis.

Q5.7

Running Total with SUM() OVER()

Scenario: Calculate a running/cumulative total of salaries ordered from lowest to highest.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
SELECT
name,
dept,
salary,
SUM(salary) OVER (
ORDER BY salary
ROWS BETWEEN UNBOUNDED PRECEDING
AND CURRENT ROW
) AS running_total,
SUM(salary) OVER () AS grand_total
FROM employees
ORDER BY salary;
```

Output / Result Set:

name	dept	salary	running_total	grand_total
Bob	HR	45000	45000	500000
Ethan	HR	45000	90000	500000
Hannah	Marketing	48000	138000	500000
George	Finance	55000	193000	500000
Diana	Finance	60000	253000	500000
Alice	IT	75000	328000	500000
Fatima	IT	82000	410000	500000
Charlie	IT	90000	500000	500000

Explanation: SUM() OVER() with ORDER BY creates a running total. ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW means: include all rows from the start up to the current row.

★ **Key Point:** SUM() OVER() without ORDER BY gives the grand total for every row — useful for calculating percentages.

NTILE() – Divide Employees into Quartiles

Scenario: Divide all employees into 4 equal salary groups (quartiles) for compensation benchmarking.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
SELECT
name,
dept,
salary,
NTILE(4) OVER (
ORDER BY salary DESC
) AS quartile,
CASE NTILE(4) OVER (ORDER BY salary DESC)
WHEN 1 THEN 'Top 25% (Q1)'
WHEN 2 THEN 'Upper Mid (Q2)'
WHEN 3 THEN 'Lower Mid (Q3)'
WHEN 4 THEN 'Bottom 25% (Q4)'
END AS quartile_label
FROM employees
ORDER BY salary DESC;
```

Output / Result Set:

name	dept	salary	quartile	quartile_label
Charlie	IT	90000	1	Top 25% (Q1)
Fatima	IT	82000	1	Top 25% (Q1)
Alice	IT	75000	2	Upper Mid (Q2)
Diana	Finance	60000	2	Upper Mid (Q2)
George	Finance	55000	3	Lower Mid (Q3)
Hannah	Marketing	48000	3	Lower Mid (Q3)
Bob	HR	45000	4	Bottom 25% (Q4)

name	dept	salary	quartile	quartile_label
Ethan	HR	45000	4	Bottom 25% (Q4)

Explanation: NTILE(n) divides rows into n roughly equal groups and assigns a group number (1 to n). With 8 employees and NTILE(4), each group gets 2 employees.

★ **Key Point:** If rows don't divide evenly, earlier groups get one extra row. NTILE(4) for 9 rows: groups 1 gets 3, groups 2-4 get 2.

Q5.9

Percentage Contribution Using SUM() OVER()

Scenario: Calculate what percentage of the total salary each department accounts for.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
SELECT
dept,
SUM(salary) AS dept_total,
SUM(SUM(salary)) OVER()
AS grand_total,
ROUND(
100.0 * SUM(salary)
/ SUM(SUM(salary)) OVER(),
2
) AS pct_of_total
FROM employees
GROUP BY dept
ORDER BY dept_total DESC;
```

Output / Result Set:

dept	dept_total	grand_total	pct_of_total
IT	247000	500000	49.4
Finance	115000	500000	23.0

dept	dept_total	grand_total	pct_of_total
HR	90000	500000	18.0
Marketing	48000	500000	9.6

Explanation: SUM(SUM(salary)) OVER() is a nested aggregate — the inner SUM groups by dept, the outer SUM() OVER() totals those grouped sums without collapsing rows.

★ **Key Point:** This nested aggregate pattern (aggregate inside a window function) is used for computing percentages and ratios.

Q5.10

FIRST_VALUE() and LAST_VALUE() in a Partition

Scenario: For each employee, show the highest and lowest salary within their own department.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
SELECT
name,
dept,
salary,
FIRST_VALUE(salary) OVER (
PARTITION BY dept
ORDER BY salary DESC
) AS dept_highest_salary,
FIRST_VALUE(salary) OVER (
PARTITION BY dept
ORDER BY salary ASC
) AS dept_lowest_salary
FROM employees
ORDER BY dept, salary DESC;
```

Output / Result Set:

name	dept	salary	dept_highest_salary	dept_lowest_salary
Diana	Finance	60000	60000	55000
George	Finance	55000	60000	55000
Bob	HR	45000	45000	45000
Ethan	HR	45000	45000	45000
Charlie	IT	90000	90000	75000
Fatima	IT	82000	90000	75000
Alice	IT	75000	90000	75000
Hannah	Marketing	48000	48000	48000

Explanation: FIRST_VALUE gets the first value in the sorted window. Ordering DESC gives the max; ordering ASC gives the min. Every row shows the dept-level max and min without reducing rows.

★ **Key Point:** LAST_VALUE requires ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING to work correctly — otherwise it only looks to the current row.

CHAPTER 06

String Functions

UPPER, LOWER, TRIM, CONCAT, SUBSTRING, LEN, CHARINDEX, REPLACE, LEFT, RIGHT, STUFF.

Q6.1

Standardize Name Case with UPPER() and LOWER()

Scenario: Employee names are stored inconsistently (uppercase, lowercase, mixed). Standardize them for reporting.

Table: employees

emp_id	first_name	last_name	email	phone	dept
1	Alice	Sharma	alice.sharma@gmail.com	9876543210	it
2	bob	Kumar	BOB.KUMAR@COMPANY.COM	9812345678	hr
3	Charlie	Singh	charlie.singh@yahoo.com	NULL	IT
4	diana	Verma	diana_verma@hotmail.com	9900112233	finance
5	Ethan	Nair	ethan.nair@gmail.com	9988776655	it
6	FATIMA	Khan	fatima.khan@company.com	9876001122	IT

SQL Query:

```
SELECT
emp_id,
UPPER(TRIM(first_name)) AS first_name_clean,
UPPER(TRIM(last_name)) AS last_name_clean,
LOWER(email) AS email_clean,
UPPER(dept) AS dept_clean
FROM employees
ORDER BY emp_id;
```

Output / Result Set:

emp_id	first_name_clean	last_name_clean	email_clean	dept_clean
1	ALICE	SHARMA	alice.sharma@gmail.com	IT
2	BOB	KUMAR	bob.kumar@company.com	HR

emp_id	first_name_clean	last_name_clean	email_clean	dept_clean
3	CHARLIE	SINGH	charlie.singh@yahoo.com	IT
4	DIANA	VERMA	diana_verma@hotmail.com	FINANCE
5	ETHAN	NAIR	ethan.nair@gmail.com	IT
6	FATIMA	KHAN	fatima.khan@company.com	IT

Explanation: UPPER() converts to uppercase, LOWER() to lowercase. TRIM() removes leading and trailing spaces. Chaining UPPER(TRIM(col)) cleans and standardizes in one step.

★ **Key Point:** Always TRIM before comparing strings — 'alice' = ' alice ' is FALSE in SQL even though they look similar.

Q6.2

Build Full Name Using CONCAT() and String Operators

Scenario: Create a display name column by combining first name, last name, and department.

Table: employees

emp_id	first_name	last_name	email	phone	dept
1	Alice	Sharma	alice.sharma@gmail.com	9876543210	it
2	bob	Kumar	BOB.KUMAR@COMPANY.COM	9812345678	hr
3	Charlie	Singh	charlie.singh@yahoo.com	NULL	IT
4	diana	Verma	diana_verma@hotmail.com	9900112233	finance
5	Ethan	Nair	ethan.nair@gmail.com	9988776655	it
6	FATIMA	Khan	fatima.khan@company.com	9876001122	IT

SQL Query:

```
SELECT
emp_id,
CONCAT(
UPPER(TRIM(first_name)),
' ',
UPPER(TRIM(last_name))
```

```

) AS full_name,
CONCAT(
UPPER(TRIM(first_name)),
' (',
UPPER(dept),
')'
) AS display_name
FROM employees
ORDER BY emp_id;

```

Output / Result Set:

emp_id	full_name	display_name
1	ALICE SHARMA	ALICE (IT)
2	BOB KUMAR	BOB (HR)
3	CHARLIE SINGH	CHARLIE (IT)
4	DIANA VERMA	DIANA (FINANCE)
5	ETHAN NAIR	ETHAN (IT)
6	FATIMA KHAN	FATIMA (IT)

Explanation: CONCAT() joins multiple strings. NULL + any string = NULL in many databases, so use CONCAT (NULL-safe) rather than the + operator for string joining.

★ **Key Point:** In MySQL use CONCAT_WS(' ', first_name, last_name) — the first argument is the separator, automatically handles NULLs.

Q6.3

Extract Domain from Email Using SUBSTRING and CHARINDEX

Scenario: Extract the email domain (e.g., 'gmail.com') from each employee's email address.

Table: employees

emp_id	first_name	last_name	email	phone	dept
1	Alice	Sharma	alice.sharma@gmail.com	9876543210	it
2	bob	Kumar	BOB.KUMAR@COMPANY.COM	9812345678	hr
3	Charlie	Singh	charlie.singh@yahoo.com	NULL	IT
4	diana	Verma	diana_verma@hotmail.com	9900112233	finance
5	Ethan	Nair	ethan.nair@gmail.com	9988776655	it

emp_id	first_name	last_name	email	phone	dept
6	FATIMA	Khan	fatima.khan@company.com	9876001122	IT

SQL Query:

```
SELECT
emp_id,
email,
SUBSTRING(
email,
CHARINDEX('@', email) + 1,
LEN(email)
) AS email_domain
FROM employees
ORDER BY emp_id;
```

Output / Result Set:

emp_id	email	email_domain
1	alice.sharma@gmail.com	gmail.com
2	BOB.KUMAR@COMPANY.COM	COMPANY.COM
3	charlie.singh@yahoo.com	yahoo.com
4	diana_verma@hotmail.com	hotmail.com
5	ethan.nair@gmail.com	gmail.com
6	fatima.khan@company.com	company.com

Explanation: CHARINDEX('@', email) finds the position of '@'. Adding 1 moves past it. SUBSTRING(string, start, length) extracts from that position to the end.

★ **Key Point:** MySQL equivalent: SUBSTRING_INDEX(email, '@', -1) — much simpler for this specific use case.

Q6.4

Find Employees Using Company Email Domain

Scenario: Find all employees who use the company email (domain = 'company.com').

Table: employees

emp_id	first_name	last_name	email	phone	dept
1	Alice	Sharma	alice.sharma@gmail.com	9876543210	it
2	bob	Kumar	BOB.KUMAR@COMPANY.COM	9812345678	hr
3	Charlie	Singh	charlie.singh@yahoo.com	NULL	IT

emp_id	first_name	last_name	email	phone	dept
4	diana	Verma	diana_verma@hotmail.com	9900112233	finance
5	Ethan	Nair	ethan.nair@gmail.com	9988776655	it
6	FATIMA	Khan	fatima.khan@company.com	9876001122	IT

SQL Query:

```
SELECT
emp_id,
CONCAT(UPPER(TRIM(first_name)),
' ',
UPPER(TRIM(last_name))) AS full_name,
LOWER(email) AS email
FROM employees
WHERE LOWER(email) LIKE '%@company.com'
ORDER BY emp_id;
```

Output / Result Set:

emp_id	full_name	email
2	BOB KUMAR	bob.kumar@company.com
6	FATIMA KHAN	fatima.khan@company.com

Explanation: LOWER(email) normalizes case before LIKE comparison. '%@company.com' matches anything ending in '@company.com'. The % wildcard matches any prefix.

★ **Key Point:** LIKE searches are often case-insensitive on MySQL and case-sensitive on PostgreSQL/SQL Server. Use LOWER() to be safe.

Q6.5

String Length and Filtering by Name Length

Scenario: Find employees whose full name (first + last) is longer than 12 characters.

Table: employees

emp_id	first_name	last_name	email	phone	dept
1	Alice	Sharma	alice.sharma@gmail.com	9876543210	it
2	bob	Kumar	BOB.KUMAR@COMPANY.COM	9812345678	hr
3	Charlie	Singh	charlie.singh@yahoo.com	NULL	IT

emp_id	first_name	last_name	email	phone	dept
4	diana	Verma	diana_verma@hotmail.com	9900112233	finance
5	Ethan	Nair	ethan.nair@gmail.com	9988776655	it
6	FATIMA	Khan	fatima.khan@company.com	9876001122	IT

SQL Query:

```
SELECT
emp_id,
TRIM(first_name) AS first_name,
TRIM(last_name) AS last_name,
LEN(TRIM(first_name)
+ ' ' +
TRIM(last_name)) AS full_name_length
FROM employees
WHERE LEN(TRIM(first_name)
+ ' ' +
TRIM(last_name)) > 12
ORDER BY full_name_length DESC;
```

Output / Result Set:

emp_id	first_name	last_name	full_name_length
3	Charlie	Singh	13
4	diana	Verma	11

Explanation: LEN() (SQL Server) or LENGTH() (MySQL/PostgreSQL) counts characters. Combining TRIM ensures leading/trailing spaces don't inflate the count.

★ **Key Point:** LEN in SQL Server excludes trailing spaces. DATALENGTH counts bytes including spaces — use LEN for character count.

Q6.6

REPLACE() – Clean Up Phone Number Format

Scenario: Phone numbers are stored with dashes (e.g., 987-654-3210). Remove dashes for clean numeric storage.

Table: contacts

id	name	phone
1	Alice	987-654-3210
2	Bob	981-234-5678
3	Charlie	NULL
4	Diana	990-011-2233

SQL Query:

```
SELECT
id,
name,
phone AS original_phone,
REPLACE(phone, '-', '')
AS cleaned_phone,
LEN(REPLACE(phone, '-', ''))
AS digit_count
FROM contacts
WHERE phone IS NOT NULL
ORDER BY id;
```

Output / Result Set:

id	name	original_phone	cleaned_phone	digit_count
1	Alice	987-654-3210	9876543210	10
2	Bob	981-234-5678	9812345678	10
4	Diana	990-011-2233	9900112233	10

Explanation: REPLACE(string, old, new) replaces all occurrences of 'old' with 'new'.
REPLACE(phone, '-', '') removes all dashes by replacing them with empty string.

★ **Key Point: Chain REPLACE for multiple characters: REPLACE(REPLACE(phone, '-', ''), ' ');** removes both dashes and spaces.

Q6.7

LEFT() and RIGHT() – Extract Parts of a String

Scenario: Extract the first 3 characters of department as a code, and last 4 digits of phone as PIN.

Table: employees

emp_id	first_name	last_name	email	phone	dept
1	Alice	Sharma	alice.sharma@gmail.com	9876543210	it
2	bob	Kumar	BOB.KUMAR@COMPANY.COM	9812345678	hr
3	Charlie	Singh	charlie.singh@yahoo.com	NULL	IT
4	diana	Verma	diana_verma@hotmail.com	9900112233	finance
5	Ethan	Nair	ethan.nair@gmail.com	9988776655	it

emp_id	first_name	last_name	email	phone	dept
6	FATIMA	Khan	fatima.khan@company.com	9876001122	IT

SQL Query:

```
SELECT
emp_id,
UPPER(TRIM(first_name)) AS name,
LEFT(UPPER(dept), 3) AS dept_code,
RIGHT(phone, 4) AS phone_last4
FROM employees
WHERE phone IS NOT NULL
ORDER BY emp_id;
```

Output / Result Set:

emp_id	name	dept_code	phone_last4
1	ALICE	IT	3210
2	BOB	HR	5678
4	DIANA	FIN	2233
5	ETHAN	IT	6655
6	FATIMA	IT	1122

Explanation: LEFT(str, n) returns the first n characters. RIGHT(str, n) returns the last n characters. Useful for extracting codes, suffixes, or partial identifiers.

★ **Key Point:** SUBSTRING(str, 1, 3) = LEFT(str, 3). SUBSTRING(str, LEN(str)-3, 4) = RIGHT(str, 4). Use LEFT/RIGHT for readability.

Q6.8

STUFF() / INSERT() – Replace Part of a String

Scenario: Mask the middle digits of phone numbers for privacy (show first 3 and last 2 digits only).

Table: employees

emp_id	first_name	last_name	email	phone	dept
1	Alice	Sharma	alice.sharma@gmail.com	9876543210	it
2	bob	Kumar	BOB.KUMAR@COMPANY.COM	9812345678	hr
3	Charlie	Singh	charlie.singh@yahoo.com	NULL	IT
4	diana	Verma	diana_verma@hotmail.com	9900112233	finance

emp_id	first_name	last_name	email	phone	dept
5	Ethan	Nair	ethan.nair@gmail.com	9988776655	it
6	FATIMA	Khan	fatima.khan@company.com	9876001122	IT

SQL Query:

```
-- SQL Server: STUFF(string, start, length, replacement)
SELECT
emp_id,
UPPER(TRIM(first_name)) AS name,
phone AS original,
STUFF(phone, 4, 5, 'XXXXX') AS masked_phone
FROM employees
WHERE phone IS NOT NULL
ORDER BY emp_id;
```

Output / Result Set:

emp_id	name	original	masked_phone
1	ALICE	9876543210	987XXXXX10
2	BOB	9812345678	981XXXXX78
4	DIANA	9900112233	990XXXXX33
5	ETHAN	9988776655	998XXXXX55
6	FATIMA	9876001122	987XXXXX22

Explanation: STUFF(string, start_pos, length_to_remove, replacement) deletes 'length' characters starting at 'start_pos' and inserts the replacement string there.

★ **Key Point: MySQL equivalent: INSERT(phone, 4, 5, 'XXXXX'). Use this for data masking and PII protection in reports.**

Q6.9

CHARINDEX / PATINDEX – Find Position of a Character

Scenario: Extract the username part (before @) from each employee's email address.

Table: employees

emp_id	first_name	last_name	email	phone	dept
1	Alice	Sharma	alice.sharma@gmail.com	9876543210	it
2	bob	Kumar	BOB.KUMAR@COMPANY.COM	9812345678	hr
3	Charlie	Singh	charlie.singh@yahoo.com	NULL	IT

emp_id	first_name	last_name	email	phone	dept
4	diana	Verma	diana_verma@hotmail.com	9900112233	finance
5	Ethan	Nair	ethan.nair@gmail.com	9988776655	it
6	FATIMA	Khan	fatima.khan@company.com	9876001122	IT

SQL Query:

```
SELECT
emp_id,
LOWER(email) AS email,
CHARINDEX('@', email) AS at_position,
LEFT(
  LOWER(email),
  CHARINDEX('@', email) - 1
) AS username
FROM employees
ORDER BY emp_id;
```

Output / Result Set:

emp_id	email	at_position	username
1	alice.sharma@gmail.com	13	alice.sharma
2	bob.kumar@company.com	10	bob.kumar
3	charlie.singh@yahoo.com	14	charlie.singh
4	diana_verma@hotmail.com	12	diana_verma
5	ethan.nair@gmail.com	11	ethan.nair
6	fatima.khan@company.com	12	fatima.khan

Explanation: CHARINDEX('@', email) finds the position of '@'. LEFT(email, pos - 1) extracts everything before it. This gives the username portion of each email.

★ **Key Point:** MySQL uses LOCATE('@', email) instead of CHARINDEX. PostgreSQL uses POSITION('@' IN email) or STRPOS(email, '@').

Q6.10

Format Employee ID with Leading Zeros

Scenario: Employee IDs need to be formatted as 5-digit strings with leading zeros (e.g., 1 becomes 'EMP00001').

Table: employees

emp_id	first_name	last_name	email	phone	dept
1	Alice	Sharma	alice.sharma@gmail.com	9876543210	it
2	bob	Kumar	BOB.KUMAR@COMPANY.COM	9812345678	hr
3	Charlie	Singh	charlie.singh@yahoo.com	NULL	IT
4	diana	Verma	diana_verma@hotmail.com	9900112233	finance
5	Ethan	Nair	ethan.nair@gmail.com	9988776655	it
6	FATIMA	Khan	fatima.khan@company.com	9876001122	IT

SQL Query:

```
SELECT
emp_id,
UPPER(TRIM(first_name)) AS name,
'EMP' + RIGHT(
'00000' + CAST(emp_id AS VARCHAR(5)),
5
) AS formatted_emp_id
FROM employees
ORDER BY emp_id;
```

Output / Result Set:

emp_id	name	formatted_emp_id
1	ALICE	EMP00001
2	BOB	EMP00002
3	CHARLIE	EMP00003
4	DIANA	EMP00004
5	ETHAN	EMP00005
6	FATIMA	EMP00006

Explanation: Prepend '00000' + emp_id, then take the RIGHT 5 characters. This always gives exactly 5 digits regardless of the original number's length.

★ **Key Point:** MySQL: `LPAD(emp_id, 5, '0')` — much simpler. SQL Server: `RIGHT('00000' + CAST(id AS VARCHAR), 5)`.

CHAPTER 07

Date & Time Functions

GETDATE, DATEADD, DATEDIFF, FORMAT, YEAR/MONTH/DAY, DATEPART, DATENAME.

Q7.1

Find Employees Who Joined in the Current Year

Scenario: HR wants a list of all employees hired in the current calendar year.

Table: employees

emp_id	name	dept	salary	hire_date	dob
1	Alice	IT	75000	2022-03-15	1995-06-10
2	Bob	HR	45000	2021-07-01	1990-11-25
3	Charlie	IT	90000	2020-01-10	1988-03-18
4	Diana	Finance	60000	2023-05-20	1993-08-07
5	Ethan	HR	42000	2022-11-30	1997-01-14
6	Fatima	IT	82000	2021-04-18	1994-09-30
7	George	Finance	55000	2019-08-25	1985-12-05

SQL Query:

```
SELECT
emp_id,
name,
dept,
hire_date
FROM employees
WHERE YEAR(hire_date) = YEAR(GETDATE())
ORDER BY hire_date;
```

Output / Result Set:

emp_id	name	dept	hire_date
4	Diana	Finance	2023-05-20

Explanation: YEAR(date) extracts the year portion. GETDATE() returns today's date and time. Comparing YEAR(hire_date) = YEAR(GETDATE()) finds this year's hires.

★ **Key Point: Avoid: WHERE hire_date >= '2023-01-01' — hardcoded years break next year. Use YEAR(GETDATE()) for dynamic queries.**

Q7.2

Calculate Employee Tenure (Years of Service)

Scenario: Calculate how many full years each employee has worked in the company.

Table: employees

emp_id	name	dept	salary	hire_date	dob
1	Alice	IT	75000	2022-03-15	1995-06-10
2	Bob	HR	45000	2021-07-01	1990-11-25
3	Charlie	IT	90000	2020-01-10	1988-03-18
4	Diana	Finance	60000	2023-05-20	1993-08-07
5	Ethan	HR	42000	2022-11-30	1997-01-14
6	Fatima	IT	82000	2021-04-18	1994-09-30
7	George	Finance	55000	2019-08-25	1985-12-05

SQL Query:

```
SELECT
emp_id,
name,
hire_date,
DATEDIFF(YEAR, hire_date, GETDATE())
AS years_of_service,
DATEDIFF(DAY, hire_date, GETDATE())
AS days_of_service
FROM employees
ORDER BY years_of_service DESC;
```

Output / Result Set:

emp_id	name	hire_date	years_of_service	days_of_service
7	George	2019-08-25	4	1385
3	Charlie	2020-01-10	3	1243
2	Bob	2021-07-01	2	727
6	Fatima	2021-04-18	2	801
1	Alice	2022-03-15	1	456
5	Ethan	2022-11-30	1	187
4	Diana	2023-05-20	0	26

Explanation: DATEDIFF(YEAR, start, end) calculates the difference in years. DATEDIFF(DAY, start, end) gives days. Note: DATEDIFF(YEAR) only compares year components, not full 365-day periods.

★ Key Point: For exact years including months: **DATEDIFF(MONTH, hire_date, GETDATE()) / 12.0** gives decimal years.

DATEADD – Calculate Contract Expiry Date

Scenario: Each employee has a 2-year contract from their hire date. Calculate when each contract expires.

Table: employees

emp_id	name	dept	salary	hire_date	dob
1	Alice	IT	75000	2022-03-15	1995-06-10
2	Bob	HR	45000	2021-07-01	1990-11-25
3	Charlie	IT	90000	2020-01-10	1988-03-18
4	Diana	Finance	60000	2023-05-20	1993-08-07
5	Ethan	HR	42000	2022-11-30	1997-01-14
6	Fatima	IT	82000	2021-04-18	1994-09-30
7	George	Finance	55000	2019-08-25	1985-12-05

SQL Query:

```
SELECT
emp_id,
name,
hire_date,
DATEADD(YEAR, 2, hire_date)
AS contract_expiry,
DATEDIFF(DAY,
GETDATE(),
DATEADD(YEAR, 2, hire_date)
) AS days_remaining
FROM employees
ORDER BY contract_expiry;
```

Output / Result Set:

emp_id	name	hire_date	contract_expiry	days_remaining
3	Charlie	2020-01-10	2022-01-10	-529
7	George	2019-08-25	2021-08-25	-667
2	Bob	2021-07-01	2023-07-01	21
6	Fatima	2021-04-18	2023-04-18	-66
1	Alice	2022-03-15	2024-03-15	259
5	Ethan	2022-11-30	2024-11-30	519
4	Diana	2023-05-20	2025-05-20	700

Explanation: DATEADD(unit, number, date) adds a specified amount to a date. Negative days_remaining means the contract has already expired.

★ Key Point: Units: YEAR, MONTH, DAY, HOUR, MINUTE, SECOND. DATEADD(MONTH, 6, date) adds 6 months.

Q7.4

Extract Date Parts – Month, Year, Day of Week

Scenario: Create a date breakdown report showing month hired, quarter, and day of week for each employee.

Table: employees

emp_id	name	dept	salary	hire_date	dob
1	Alice	IT	75000	2022-03-15	1995-06-10
2	Bob	HR	45000	2021-07-01	1990-11-25
3	Charlie	IT	90000	2020-01-10	1988-03-18
4	Diana	Finance	60000	2023-05-20	1993-08-07
5	Ethan	HR	42000	2022-11-30	1997-01-14
6	Fatima	IT	82000	2021-04-18	1994-09-30
7	George	Finance	55000	2019-08-25	1985-12-05

SQL Query:

```
SELECT
emp_id,
name,
hire_date,
DAY(hire_date) AS hire_day,
MONTH(hire_date) AS hire_month,
YEAR(hire_date) AS hire_year,
DATENAME(WEEKDAY, hire_date) AS day_of_week,
DATEPART(QUARTER, hire_date) AS quarter
FROM employees
ORDER BY hire_date;
```

Output / Result Set:

emp_id	name	hire_date	hire_day	hire_month	hire_year	day_of_week	quarter
7	George	2019-08-25	25	8	2019	Sunday	3
3	Charlie	2020-01-10	10	1	2020	Friday	1
2	Bob	2021-07-01	1	7	2021	Thursday	3
6	Fatima	2021-04-18	18	4	2021	Sunday	2
1	Alice	2022-03-15	15	3	2022	Tuesday	1
5	Ethan	2022-11-30	30	11	2022	Wednesday	4

emp_id	name	hire_date	hire_day	hire_month	hire_year	day_of_week	quarter
4	Diana	2023-05-20	20	5	2023	Saturday	2

Explanation: DAY(), MONTH(), YEAR() extract individual components. DATENAME() returns a text name. DATEPART() returns a numeric value. Both DATENAME and DATEPART accept the same date parts.

★ **Key Point:** MySQL uses DAYOFWEEK(), DAYNAME(), QUARTER() — slightly different syntax but same concept.

Q7.5

Find Employees with Birthdays in the Next 30 Days

Scenario: HR wants to send birthday wishes. Find employees whose birthday falls within the next 30 days.

Table: employees

emp_id	name	dept	salary	hire_date	dob
1	Alice	IT	75000	2022-03-15	1995-06-10
2	Bob	HR	45000	2021-07-01	1990-11-25
3	Charlie	IT	90000	2020-01-10	1988-03-18
4	Diana	Finance	60000	2023-05-20	1993-08-07
5	Ethan	HR	42000	2022-11-30	1997-01-14
6	Fatima	IT	82000	2021-04-18	1994-09-30
7	George	Finance	55000	2019-08-25	1985-12-05

SQL Query:

```
SELECT
emp_id,
name,
dob,
DATEFROMPARTS(
YEAR(GETDATE()),
MONTH(dob),
DAY(dob)
) AS birthday_this_year,
DATEDIFF(DAY,
GETDATE(),
DATEFROMPARTS(
YEAR(GETDATE()),
MONTH(dob),
DAY(dob)
)
)
```

```

) AS days_until_birthday
FROM employees
WHERE
DATEDIFF(DAY, GETDATE(),
DATEFROMPARTS(YEAR(GETDATE()),
MONTH(dob), DAY(dob)))
BETWEEN 0 AND 30
ORDER BY days_until_birthday;

```

Output / Result Set:

emp_id	name	dob	birthday_this_year	days_until_birthday
4	Diana	1993-08-07	2023-08-07	18
3	Charlie	1988-03-18	2023-03-18	17

Explanation: DATEFROMPARTS(year, month, day) builds a date from parts. We rebuild each employee's birthday with the current year, then find those within the next 30 days.

★ **Key Point: Edge case:** This doesn't handle employees whose birthday has already passed this year. For full year-wrap handling, add a CASE or check next year's date if current year's is past.

Q7.6

Calculate Age from Date of Birth

Scenario: Calculate the exact age of each employee in years.

Table: employees

emp_id	name	dept	salary	hire_date	dob
1	Alice	IT	75000	2022-03-15	1995-06-10
2	Bob	HR	45000	2021-07-01	1990-11-25
3	Charlie	IT	90000	2020-01-10	1988-03-18
4	Diana	Finance	60000	2023-05-20	1993-08-07
5	Ethan	HR	42000	2022-11-30	1997-01-14
6	Fatima	IT	82000	2021-04-18	1994-09-30
7	George	Finance	55000	2019-08-25	1985-12-05

SQL Query:

```

SELECT
emp_id,
name,
dob,
DATEDIFF(YEAR, dob, GETDATE())
- CASE
WHEN MONTH(GETDATE()) < MONTH(dob)

```

```

OR (MONTH(GETDATE()) = MONTH(dob)
AND DAY(GETDATE()) < DAY(dob))
THEN 1
ELSE 0
END AS exact_age
FROM employees
ORDER BY exact_age DESC;

```

Output / Result Set:

emp_id	name	dob	exact_age
7	George	1985-12-05	37
3	Charlie	1988-03-18	35
2	Bob	1990-11-25	32
4	Diana	1993-08-07	29
6	Fatima	1994-09-30	28
1	Alice	1995-06-10	27
5	Ethan	1997-01-14	26

Explanation: DATEDIFF(YEAR) only compares years (not if birthday has passed). The CASE subtracts 1 if the birthday hasn't occurred yet this year, giving the exact current age.

★ **Key Point:** This is the correct way to calculate age in SQL Server. Simply DATEDIFF(YEAR, dob, GETDATE()) overestimates age for those who haven't had their birthday yet.

Q7.7

FORMAT() – Display Dates in Custom Format

Scenario: Display hire dates in user-friendly formats for different reports.

Table: employees

emp_id	name	dept	salary	hire_date	dob
1	Alice	IT	75000	2022-03-15	1995-06-10
2	Bob	HR	45000	2021-07-01	1990-11-25
3	Charlie	IT	90000	2020-01-10	1988-03-18
4	Diana	Finance	60000	2023-05-20	1993-08-07
5	Ethan	HR	42000	2022-11-30	1997-01-14
6	Fatima	IT	82000	2021-04-18	1994-09-30
7	George	Finance	55000	2019-08-25	1985-12-05

SQL Query:

```

SELECT
emp_id,

```

```

name,
hire_date,
FORMAT(hire_date, 'dd-MMM-yyyy')
AS formatted_date,
FORMAT(hire_date, 'MMMM dd, yyyy')
AS long_format,
FORMAT(hire_date, 'yyyy/MM/dd')
AS iso_format
FROM employees
ORDER BY hire_date;

```

Output / Result Set:

emp_id	name	formatted_date	long_format	iso_format
7	George	25-Aug-2019	August 25, 2019	2019/08/25
3	Charlie	10-Jan-2020	January 10, 2020	2020/01/10
2	Bob	01-Jul-2021	July 01, 2021	2021/07/01
6	Fatima	18-Apr-2021	April 18, 2021	2021/04/18
1	Alice	15-Mar-2022	March 15, 2022	2022/03/15
5	Ethan	30-Nov-2022	November 30, 2022	2022/11/30
4	Diana	20-May-2023	May 20, 2023	2023/05/20

Explanation: FORMAT(date, format_string) formats dates for display. Common codes: dd=day, MM=month number, MMM=month abbr, MMMM=full month, yyyy=4-digit year.

★ **Key Point: FORMAT is for display only — don't use it in WHERE clauses or comparisons. Always store and compare dates in their native DATE type.**

Q7.8

Find Employees Who Joined on Weekends

Scenario: Audit: find any employees whose hire date was a Saturday or Sunday (potential data entry error).

Table: employees

emp_id	name	dept	salary	hire_date	dob
1	Alice	IT	75000	2022-03-15	1995-06-10
2	Bob	HR	45000	2021-07-01	1990-11-25
3	Charlie	IT	90000	2020-01-10	1988-03-18
4	Diana	Finance	60000	2023-05-20	1993-08-07
5	Ethan	HR	42000	2022-11-30	1997-01-14
6	Fatima	IT	82000	2021-04-18	1994-09-30
7	George	Finance	55000	2019-08-25	1985-12-05

SQL Query:

```
SELECT
emp_id,
name,
hire_date,
DATENAME(WEEKDAY, hire_date) AS day_of_week
FROM employees
WHERE DATEPART(WEEKDAY, hire_date) IN (1, 7)
ORDER BY hire_date;

-- Note: In SQL Server, 1=Sunday, 7=Saturday by default
-- (depends on SET DATEFIRST setting)
```

Output / Result Set:

emp_id	name	hire_date	day_of_week
7	George	2019-08-25	Sunday
6	Fatima	2021-04-18	Sunday
4	Diana	2023-05-20	Saturday

Explanation: DATEPART(WEEKDAY, date) returns a number for the day of the week (1=Sunday to 7=Saturday in SQL Server). Filtering for 1 and 7 finds weekend dates.

★ **Key Point:** MySQL: DAYOFWEEK() returns 1=Sunday,7=Saturday. PostgreSQL: EXTRACT(DOW FROM date) returns 0=Sunday,6=Saturday.

Q7.9

Orders Within Last N Days (Dynamic Date Filter)

Scenario: Show all orders placed in the last 90 days without hardcoding any date.

Table: orders

order_id	customer	amount	order_date
1	Alice	5000	2024-06-01
2	Bob	3200	2024-05-15
3	Charlie	8900	2024-03-20
4	Diana	1500	2024-01-10
5	Ethan	4700	2023-11-05

SQL Query:

```
SELECT
order_id,
customer,
amount,
order_date,
DATEDIFF(DAY, order_date, GETDATE())
```

```
AS days_ago
FROM orders
WHERE order_date >= DATEADD(DAY, -90, GETDATE())
ORDER BY order_date DESC;
```

Output / Result Set:

order_id	customer	amount	order_date	days_ago
1	Alice	5000	2024-06-01	14
2	Bob	3200	2024-05-15	30
3	Charlie	8900	2024-03-20	87

Explanation: DATEADD(DAY, -90, GETDATE()) computes the date exactly 90 days ago. Using >= instead of = is critical — it captures all orders on or after that date.

★ **Key Point:** Always use >= with dates, never = alone. An exact equality match on DATETIME may miss records stored at different times on the same day.

Q7.10

Group Data by Month-Year for Trend Analysis

Scenario: Show monthly revenue totals for trend analysis.

Table: sales

sale_id	product	amount	sale_date
1	Laptop	85000	2024-01-05
2	Phone	45000	2024-01-18
3	Monitor	22000	2024-02-10
4	Laptop	85000	2024-02-28
5	Mouse	1500	2024-03-08
6	Phone	45000	2024-03-22
7	Laptop	85000	2024-03-30

SQL Query:

```
SELECT
FORMAT(sale_date, 'yyyy-MM') AS sale_month,
DATENAME(MONTH, sale_date) AS month_name,
COUNT(sale_id) AS total_orders,
SUM(amount) AS total_revenue,
ROUND(AVG(CAST(amount AS FLOAT)),0)
AS avg_order_value
FROM sales
GROUP BY
FORMAT(sale_date, 'yyyy-MM'),
DATENAME(MONTH, sale_date)
```

```
ORDER BY sale_month;
```

Output / Result Set:

sale_month	month_name	total_orders	total_revenue	avg_order_value
2024-01	January	2	130000	65000
2024-02	February	2	107000	53500
2024-03	March	3	131500	43833

Explanation: FORMAT(date, 'yyyy-MM') creates a consistent sort key. Grouping by this string ensures months are in calendar order. CAST to FLOAT ensures decimal division in AVG.

★ **Key Point:** In MySQL: DATE_FORMAT(sale_date, '%Y-%m') is the equivalent. Always include the year to avoid merging data across different years.

CHAPTER 08

NULL Handling & CASE WHEN

IS NULL, ISNULL, COALESCE, NULLIF, CASE WHEN, conditional aggregation.

Q8.1

Find All Records with NULL Values

Scenario: Audit: find employees with missing salary, bonus, or phone numbers.

Table: employees

emp_id	name	dept	salary	bonus	phone	manager_id
1	Alice	IT	75000	5000	9876543210	3
2	Bob	HR	NULL	NULL	9812345678	NULL
3	Charlie	IT	90000	NULL	NULL	NULL
4	Diana	Finance	60000	3000	9900112233	3
5	Ethan	HR	42000	1000	NULL	3
6	Fatima	IT	82000	4000	9876001122	3
7	George	Finance	NULL	2000	9911223344	NULL

SQL Query:

```
SELECT
emp_id,
name,
dept,
salary,
bonus,
phone
FROM employees
WHERE salary IS NULL
OR bonus IS NULL
OR phone IS NULL
ORDER BY emp_id;
```

Output / Result Set:

emp_id	name	dept	salary	bonus	phone
2	Bob	HR	NULL	NULL	9812345678
3	Charlie	IT	90000	NULL	NULL

emp_id	name	dept	salary	bonus	phone
5	Ethan	HR	42000	1000	NULL
6	Fatima	IT	82000	4000	9876001122
7	George	Finance	NULL	2000	9911223344

Explanation: IS NULL tests for missing values. Multiple IS NULL checks with OR catches rows where ANY of the fields is missing. NULL cannot be compared with = or !=.

★ **Key Point: Remember: NULL = NULL is FALSE in SQL. Only IS NULL returns TRUE. This is called three-valued logic (TRUE, FALSE, UNKNOWN).**

Q8.2

ISNULL / COALESCE – Replace NULL with Default Values

Scenario: Replace NULL salaries with 0 and NULL bonus with 'No Bonus' for a clean salary report.

Table: employees

emp_id	name	dept	salary	bonus	phone	manager_id
1	Alice	IT	75000	5000	9876543210	3
2	Bob	HR	NULL	NULL	9812345678	NULL
3	Charlie	IT	90000	NULL	NULL	NULL
4	Diana	Finance	60000	3000	9900112233	3
5	Ethan	HR	42000	1000	NULL	3
6	Fatima	IT	82000	4000	9876001122	3
7	George	Finance	NULL	2000	9911223344	NULL

SQL Query:

```
SELECT
emp_id,
name,
dept,
ISNULL(salary, 0) AS salary,
ISNULL(
CAST(bonus AS VARCHAR),
'No Bonus'
) AS bonus_display,
COALESCE(phone,
'Not Provided') AS phone
FROM employees
```

```
ORDER BY emp_id;
```

Output / Result Set:

emp_id	name	dept	salary	bonus_display	phone
1	Alice	IT	75000	5000	9876543210
2	Bob	HR	0	No Bonus	9812345678
3	Charlie	IT	90000	No Bonus	Not Provided
4	Diana	Finance	60000	3000	9900112233
5	Ethan	HR	42000	1000	Not Provided
6	Fatima	IT	82000	4000	9876001122
7	George	Finance	0	2000	9911223344

Explanation: ISNULL(expr, replacement) is SQL Server specific — returns replacement if expr is NULL. COALESCE(a, b, c) is ANSI standard — returns first non-NULL value from the list.

★ **Key Point:** COALESCE is preferred over ISNULL — it's portable, handles multiple fallbacks, and is ANSI SQL. COALESCE(phone, mobile, email) tries each in order.

Q8.3

CASE WHEN – Categorize Employees by Salary Band

Scenario: Classify each employee into salary categories: Junior, Mid-Level, Senior, or Lead.

Table: employees

emp_id	name	dept	salary	bonus	phone	manager_id
1	Alice	IT	75000	5000	9876543210	3
2	Bob	HR	NULL	NULL	9812345678	NULL
3	Charlie	IT	90000	NULL	NULL	NULL
4	Diana	Finance	60000	3000	9900112233	3
5	Ethan	HR	42000	1000	NULL	3
6	Fatima	IT	82000	4000	9876001122	3
7	George	Finance	NULL	2000	9911223344	NULL

SQL Query:

```
SELECT
emp_id,
name,
```

```

dept,
ISNULL(salary, 0) AS salary,
CASE
WHEN ISNULL(salary,0) = 0 THEN 'Salary Not Set'
WHEN ISNULL(salary,0) < 45000 THEN 'Junior'
WHEN ISNULL(salary,0) < 65000 THEN 'Mid-Level'
WHEN ISNULL(salary,0) < 85000 THEN 'Senior'
ELSE 'Lead'
END AS salary_band
FROM employees
ORDER BY ISNULL(salary, 0) DESC;

```

Output / Result Set:

emp_id	name	dept	salary	salary_band
3	Charlie	IT	90000	Lead
6	Fatima	IT	82000	Senior
1	Alice	IT	75000	Senior
4	Diana	Finance	60000	Mid-Level
5	Ethan	HR	42000	Junior
7	George	Finance	0	Salary Not Set
2	Bob	HR	0	Salary Not Set

Explanation: CASE WHEN evaluates conditions top to bottom and returns the first matching result. Always handle edge cases (NULL, 0) in the first WHEN, and use ELSE as a catch-all.

★ **Key Point:** CASE WHEN is evaluated in order — once a condition matches, the rest are skipped. Order conditions from most specific to least specific.

Q8.4

NULLIF – Avoid Division by Zero

Scenario: Calculate bonus as percentage of salary — avoid division by zero for employees with no salary.

Table: employees

emp_id	name	dept	salary	bonus	phone	manager_id
1	Alice	IT	75000	5000	9876543210	3
2	Bob	HR	NULL	NULL	9812345678	NULL
3	Charlie	IT	90000	NULL	NULL	NULL
4	Diana	Finance	60000	3000	9900112233	3

emp_id	name	dept	salary	bonus	phone	manager_id
5	Ethan	HR	42000	1000	NULL	3
6	Fatima	IT	82000	4000	9876001122	3
7	George	Finance	NULL	2000	9911223344	NULL

SQL Query:

```
SELECT
emp_id,
name,
ISNULL(salary, 0) AS salary,
ISNULL(bonus, 0) AS bonus,
ROUND(
100.0 * ISNULL(bonus, 0)
/ NULLIF(ISNULL(salary, 0), 0),
2
) AS bonus_pct
FROM employees
ORDER BY bonus_pct DESC;
```

Output / Result Set:

emp_id	name	salary	bonus	bonus_pct	
5	Ethan	IT	42000	1000	2.38
4	Diana	Finance	60000	3000	5.0
6	Fatima	IT	82000	4000	4.88
1	Alice	IT	75000	5000	6.67
3	Charlie	IT	90000	0	NULL
2	Bob	HR	0	0	NULL
7	George	Finance	0	2000	NULL

Explanation: NULLIF(expr, value) returns NULL if expr equals value, otherwise returns expr. NULLIF(salary, 0) converts 0 to NULL, then dividing by NULL returns NULL — avoiding the division by zero error.

★ **Key Point:** Division by zero error is common in SQL. Always wrap the denominator with NULLIF(denominator, 0).

Q8.5

Conditional Aggregation – Count by Status Using CASE WHEN

Scenario: Count how many employees have and don't have a phone number, grouped by department.

Table: employees

emp_id	name	dept	salary	bonus	phone	manager_id
1	Alice	IT	75000	5000	9876543210	3
2	Bob	HR	NULL	NULL	9812345678	NULL
3	Charlie	IT	90000	NULL	NULL	NULL
4	Diana	Finance	60000	3000	9900112233	3
5	Ethan	HR	42000	1000	NULL	3
6	Fatima	IT	82000	4000	9876001122	3
7	George	Finance	NULL	2000	9911223344	NULL

SQL Query:

```
SELECT
dept,
COUNT(*) AS total,
COUNT(CASE WHEN phone IS NOT NULL
THEN 1 END) AS has_phone,
COUNT(CASE WHEN phone IS NULL
THEN 1 END) AS missing_phone,
COUNT(CASE WHEN salary IS NULL
THEN 1 END) AS missing_salary
FROM employees
GROUP BY dept
ORDER BY dept;
```

Output / Result Set:

dept	total	has_phone	missing_phone	missing_salary
Finance	2	2	0	1
HR	2	1	1	1
IT	3	2	1	0

Explanation: SUM(CASE WHEN condition THEN 1 ELSE 0 END) counts rows where the condition is TRUE. This pivot-style reporting shows completeness metrics per department.

★ **Key Point:** This is called **Conditional Aggregation** — one of the most useful patterns in SQL. Learn it well.

Q8.6

CASE WHEN in ORDER BY – Custom Sort Order

Scenario: Sort employees: IT first, then HR, then Finance, then any other department.

Table: employees

emp_id	name	dept	salary	bonus	phone	manager_id
1	Alice	IT	75000	5000	9876543210	3
2	Bob	HR	NULL	NULL	9812345678	NULL
3	Charlie	IT	90000	NULL	NULL	NULL
4	Diana	Finance	60000	3000	9900112233	3
5	Ethan	HR	42000	1000	NULL	3
6	Fatima	IT	82000	4000	9876001122	3
7	George	Finance	NULL	2000	9911223344	NULL

SQL Query:

```
SELECT
emp_id,
name,
dept,
ISNULL(salary,0) AS salary
FROM employees
ORDER BY
CASE dept
WHEN 'IT' THEN 1
WHEN 'HR' THEN 2
WHEN 'Finance' THEN 3
ELSE 4
END,
name;
```

Output / Result Set:

emp_id	name	dept	salary
1	Alice	IT	75000
3	Charlie	IT	90000
6	Fatima	IT	82000
2	Bob	HR	0
5	Ethan	HR	42000
4	Diana	Finance	60000
7	George	Finance	0

Explanation: CASE WHEN in ORDER BY assigns numeric priority to each category. Rows are sorted by the computed priority number, not the original string value.

★ Key Point: This pattern is useful when you need to sort by business priority rather than alphabetical order.

Q8.7

CASE WHEN for Data Transformation – Gender/Status Labels

Scenario: A legacy system stores status as 1/0/NULL. Transform to readable labels in the output.

Table: employees

emp_id	name	status	notice_period
1	Alice	1	0
2	Bob	0	30
3	Charlie	1	0
4	Diana	NULL	60
5	Ethan	0	NULL

SQL Query:

```
SELECT
emp_id,
name,
CASE status
WHEN 1 THEN 'Active'
WHEN 0 THEN 'Inactive'
ELSE 'Unknown'
END AS emp_status,
CASE
WHEN notice_period IS NULL THEN 'Not Set'
WHEN notice_period = 0 THEN 'Immediate'
WHEN notice_period <= 30 THEN '30 Days'
ELSE '60+ Days'
END AS notice_label
FROM employees
ORDER BY emp_id;
```

Output / Result Set:

emp_id	name	emp_status	notice_label
1	Alice	Active	Immediate
2	Bob	Inactive	30 Days
3	Charlie	Active	Immediate
4	Diana	Unknown	60+ Days
5	Ethan	Inactive	Not Set

Explanation: CASE colname WHEN value THEN result (simple CASE) is cleaner for equality checks. CASE WHEN condition THEN result (searched CASE) handles ranges and complex conditions.

★ **Key Point:** Simple CASE: checks equality only. Searched CASE: checks any condition. Use searched CASE for NULL checks and ranges.

Q8.8

COALESCE with Multiple Fallbacks

Scenario: Contact preference: use work phone first, then mobile, then email, then 'No Contact'.

Table: contacts

id	name	work_phone	mobile	email
1	Alice	9876543210	NULL	alice@gmail.com
2	Bob	NULL	9812345678	bob@yahoo.com
3	Charlie	NULL	NULL	charlie@gmail.com
4	Diana	NULL	NULL	NULL
5	Ethan	9988776655	9911223344	ethan@gmail.com

SQL Query:

```
SELECT
id,
name,
COALESCE(
work_phone,
mobile,
email,
'No Contact Info'
) AS preferred_contact,
CASE
WHEN work_phone IS NOT NULL THEN 'Work Phone'
WHEN mobile IS NOT NULL THEN 'Mobile'
WHEN email IS NOT NULL THEN 'Email'
ELSE 'None'
END AS contact_type
FROM contacts
ORDER BY id;
```

Output / Result Set:

id	name	preferred_contact	contact_type
1	Alice	9876543210	Work Phone

id	name	preferred_contact	contact_type
2	Bob	9812345678	Mobile
3	Charlie	charlie@gmail.com	Email
4	Diana	No Contact Info	None
5	Ethan	9988776655	Work Phone

Explanation: COALESCE returns the first non-NULL value from its argument list. It's perfect for implementing priority-based fallback logic across multiple columns.

★ **Key Point:** COALESCE is ANSI SQL and works in all databases. It can take any number of arguments — invaluable for data quality scenarios.

Q8.9

Update NULL Salary with Department Average

Scenario: For employees with NULL salary, set their salary to the average salary of their department.

Table: employees

emp_id	name	dept	salary	bonus	phone	manager_id
1	Alice	IT	75000	5000	9876543210	3
2	Bob	HR	NULL	NULL	9812345678	NULL
3	Charlie	IT	90000	NULL	NULL	NULL
4	Diana	Finance	60000	3000	9900112233	3
5	Ethan	HR	42000	1000	NULL	3
6	Fatima	IT	82000	4000	9876001122	3
7	George	Finance	NULL	2000	9911223344	NULL

SQL Query:

```
UPDATE employees
SET salary = (
  SELECT ROUND(AVG(e2.salary), 0)
  FROM employees e2
  WHERE e2.dept = employees.dept
  AND e2.salary IS NOT NULL
)
WHERE salary IS NULL;

-- Verify the update:
SELECT emp_id, name, dept, salary
```

```
FROM employees
ORDER BY dept, emp_id;
```

Output / Result Set:

emp_id	name	dept	salary
4	Diana	Finance	60000
7	George	Finance	60000
2	Bob	HR	42000
5	Ethan	HR	42000
1	Alice	IT	75000
3	Charlie	IT	90000
6	Fatima	IT	82000

Explanation: The correlated subquery in the SET clause calculates the dept average excluding NULLs. The WHERE salary IS NULL ensures we only update records that need it.

★ **Key Point:** Always run **SELECT** before **UPDATE** to verify which rows will be affected. Add **BEGIN TRANSACTION** and **ROLLBACK** for safety in production.

Q8.10

Handling NULL in Aggregates and Comparisons

Scenario: Show the total compensation (salary + bonus) handling NULLs, and count non-null values.

Table: employees

emp_id	name	dept	salary	bonus	phone	manager_id
1	Alice	IT	75000	5000	9876543210	3
2	Bob	HR	NULL	NULL	9812345678	NULL
3	Charlie	IT	90000	NULL	NULL	NULL
4	Diana	Finance	60000	3000	9900112233	3
5	Ethan	HR	42000	1000	NULL	3
6	Fatima	IT	82000	4000	9876001122	3
7	George	Finance	NULL	2000	9911223344	NULL

SQL Query:

```
SELECT
emp_id,
name,
```

```

ISNULL(salary, 0) AS salary,
ISNULL(bonus, 0) AS bonus,
ISNULL(salary, 0)
+ ISNULL(bonus, 0) AS total_comp,
COUNT(*) OVER() AS total_emp,
COUNT(salary) OVER() AS emp_with_salary,
COUNT(bonus) OVER() AS emp_with_bonus
FROM employees
ORDER BY total_comp DESC;

```

Output / Result Set:

emp_id	name	salary	bonus	total_comp	total_emp	emp_with_salary	emp_with_bonus	
3	Charlie	IT	90000	0	90000	7	5	4
1	Alice	IT	75000	5000	80000	7	5	4
6	Fatima	IT	82000	4000	86000	7	5	4
4	Diana	Finance	60000	3000	63000	7	5	4
7	George	Finance	0	2000	2000	7	5	4
5	Ethan	HR	42000	1000	43000	7	5	4
2	Bob	HR	0	0	0	7	5	4

Explanation: salary + NULL = NULL. Always use ISNULL/COALESCE before arithmetic. COUNT(*) counts all rows; COUNT(column) skips NULLs — so COUNT(salary) = 5 while COUNT(*) = 7.

★ **Key Point:** Arithmetic with NULL always returns NULL. This is the most common cause of unexpected zeros or missing values in financial calculations.

CHAPTER 09

CTEs (Common Table Expressions)

Basic CTEs, multiple CTEs, recursive CTEs, CTEs with DML, pagination, and MoM analysis.

Q9.1

Basic CTE – Employees Above Average Salary

Scenario: Use a CTE to find employees earning above the company average salary.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
WITH avg_salary AS (  
  SELECT AVG(salary) AS company_avg  
  FROM employees  
)  
SELECT  
  e.emp_id,  
  e.name,  
  e.dept,  
  e.salary,  
  a.company_avg  
FROM employees e  
CROSS JOIN avg_salary a  
WHERE e.salary > a.company_avg  
ORDER BY e.salary DESC;
```

Output / Result Set:

emp_id	name	dept	salary	company_avg
3	Charlie	IT	90000	62625
6	Fatima	IT	82000	62625

emp_id	name	dept	salary	company_avg
1	Alice	IT	75000	62625
4	Diana	Finance	60000	62625

Explanation: WITH cte_name AS (query) defines a named temporary result set. The CTE is then used just like a table in the main query. It makes complex queries more readable.

★ **Key Point:** CTEs exist only for the duration of the query — they're not stored. Think of them as named subqueries that can be referenced multiple times.

Q9.2

Multiple CTEs – Chained Analysis

Scenario: Use multiple CTEs to find departments where both average salary > 50000 AND they have > 1 employee.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
WITH dept_stats AS (
  SELECT
    dept,
    COUNT(*) AS headcount,
    AVG(salary) AS avg_salary,
    SUM(salary) AS total_salary
  FROM employees
  GROUP BY dept
),
qualified_depts AS (
  SELECT dept, headcount, avg_salary
  FROM dept_stats
  WHERE avg_salary > 50000
  AND headcount > 1
)
```

```

SELECT
e.name,
e.dept,
e.salary,
q.avg_salary AS dept_avg
FROM employees e
INNER JOIN qualified_depts q
ON e.dept = q.dept
ORDER BY e.dept, e.salary DESC;

```

Output / Result Set:

name	dept	salary	dept_avg
Charlie	IT	90000	82333
Fatima	IT	82000	82333
Alice	IT	75000	82333

Explanation: Multiple CTEs are separated by commas. Each CTE can reference CTEs defined before it. The final SELECT references the last CTE or any previous ones.

★ **Key Point:** CTEs are processed in definition order. A later CTE can reference an earlier one, but not vice versa.

Q9.3

CTE for Top N Per Group

Scenario: Find the top 2 highest-paid employees per department using a CTE.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```

WITH ranked_employees AS (
SELECT
emp_id,
name,
dept,

```

```

salary,
DENSE_RANK() OVER (
PARTITION BY dept
ORDER BY salary DESC
) AS dept_rank
FROM employees
)
SELECT
emp_id,
name,
dept,
salary,
dept_rank
FROM ranked_employees
WHERE dept_rank <= 2
ORDER BY dept, dept_rank;

```

Output / Result Set:

emp_id	name	dept	salary	dept_rank
4	Diana	Finance	60000	1
7	George	Finance	55000	2
2	Bob	HR	45000	1
5	Ethan	HR	45000	1
3	Charlie	IT	90000	1
6	Fatima	IT	82000	2
8	Hannah	Marketing	48000	1

Explanation: CTE wraps the window function query. The outer SELECT filters using the computed rank. Note Bob and Ethan both get rank 1 (tied), so both appear — there's no rank 2 in HR.

★ **Key Point:** CTE + Window Function + WHERE on window result = the Top-N per group pattern. One of the most common interview patterns.

Q9.4

Recursive CTE – Employee Hierarchy

Scenario: Display the full employee hierarchy showing each employee and their level in the org chart.

Table: employees

emp_id	name	manager_id	dept
1	CEO Alice	NULL	Executive
2	VP Bob	1	IT

emp_id	name	manager_id	dept
3	VP Carol	1	HR
4	Mgr Dave	2	IT
5	Mgr Eve	2	IT
6	Eng Frank	4	IT
7	Eng Grace	4	IT
8	HR Head Henry	3	HR

SQL Query:

```

WITH RECURSIVE org_chart AS (
  -- Anchor: start with top-level (no manager)
  SELECT
    emp_id,
    name,
    manager_id,
    dept,
    1 AS level,
    CAST(name AS VARCHAR(500)) AS path
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  -- Recursive: add direct reports
  SELECT
    e.emp_id,
    e.name,
    e.manager_id,
    e.dept,
    oc.level + 1,
    oc.path + ' > ' + e.name
  FROM employees e
  INNER JOIN org_chart oc
  ON e.manager_id = oc.emp_id
)
SELECT
  emp_id,
  REPLICATE(' ', level-1) + name AS name_indented,
  level,
  path
FROM org_chart
ORDER BY path;

```

Output / Result Set:

emp_id	name_indented	level	path
1	CEO Alice	1	CEO Alice
3	VP Carol	2	CEO Alice > VP Carol
8	HR Head Henry	3	CEO Alice > VP Carol > HR Head Henry
2	VP Bob	2	CEO Alice > VP Bob
4	Mgr Dave	3	CEO Alice > VP Bob > Mgr Dave
6	Eng Frank	4	CEO Alice > VP Bob > Mgr Dave > Eng Frank
7	Eng Grace	4	CEO Alice > VP Bob > Mgr Dave > Eng Grace
5	Mgr Eve	3	CEO Alice > VP Bob > Mgr Eve

Explanation: Recursive CTE has two parts: Anchor (base case, no manager) and Recursive (joins back to itself). UNION ALL combines them. It runs until no more rows can be added.

★ **Key Point:** Recursive CTEs are the standard way to handle tree/hierarchical data in SQL. Most databases support them (SQL Server, PostgreSQL, MySQL 8+).

Q9.5

CTE to Find Duplicate Records

Scenario: Find duplicate employee records (same name and department appearing more than once).

Table: employees

emp_id	name	dept	salary
1	Alice	IT	75000
2	Bob	HR	45000
3	Alice	IT	75000
4	Charlie	IT	90000
5	Bob	HR	46000
6	Diana	Finance	60000

SQL Query:

```
WITH duplicates AS (
SELECT
  name,
  dept,
  COUNT(*) AS occurrences
FROM employees
GROUP BY name, dept)
```

```

HAVING COUNT(*) > 1
)
SELECT
e.emp_id,
e.name,
e.dept,
e.salary,
d.occurrences
FROM employees e
INNER JOIN duplicates d
ON e.name = d.name
AND e.dept = d.dept
ORDER BY e.name, e.dept, e.emp_id;

```

Output / Result Set:

emp_id	name	dept	salary	occurrences
1	Alice	IT	75000	2
3	Alice	IT	75000	2
2	Bob	HR	45000	2
5	Bob	HR	46000	2

Explanation: The CTE finds name-dept combinations with more than one occurrence. The outer JOIN retrieves all full records matching those combinations, showing complete duplicate details.

★ **Key Point:** To delete duplicates, keep only the MIN(emp_id) per group: **DELETE FROM employees WHERE emp_id NOT IN (SELECT MIN(emp_id) FROM employees GROUP BY name, dept).**

Q9.6

CTE for Running Total and Percentage

Scenario: Calculate a running total of salary and what percentage of total payroll each employee represents.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000

emp_id	name	dept	city	salary
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
WITH salary_ranked AS (
SELECT
name,
dept,
salary,
SUM(salary) OVER () AS total_payroll,
SUM(salary) OVER (
ORDER BY salary DESC
) AS running_total
FROM employees
)
SELECT
name,
dept,
salary,
running_total,
ROUND(100.0 * salary / total_payroll, 2)
AS pct_of_payroll
FROM salary_ranked
ORDER BY salary DESC;
```

Output / Result Set:

name	dept	salary	running_total	pct_of_payroll
Charlie	IT	90000	90000	18.0
Fatima	IT	82000	172000	16.4
Alice	IT	75000	247000	15.0
Diana	Finance	60000	307000	12.0
George	Finance	55000	362000	11.0
Hannah	Marketing	48000	410000	9.6
Bob	HR	45000	455000	9.0
Ethan	HR	45000	500000	9.0

Explanation: The CTE computes both grand total and running total using window functions. The outer query uses these to calculate percentages without repeating the window function logic.

★ **Key Point:** CTEs improve readability — complex window functions are defined once in the CTE and used cleanly in the main query.

Q9.7

CTE vs Subquery – Readability Comparison

Scenario: Find the department with the highest average salary. Write both with subquery and with CTE.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
-- Method 1: Subquery (harder to read)
SELECT dept, avg_salary
FROM (
  SELECT dept, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY dept
) dept_avgs
WHERE avg_salary = (
  SELECT MAX(avg_salary)
  FROM (
    SELECT AVG(salary) AS avg_salary
    FROM employees GROUP BY dept
  ) x
);

-- Method 2: CTE (clean & readable)
WITH dept_avgs AS (
  SELECT dept,
  ROUND(AVG(salary),0) AS avg_salary
  FROM employees
  GROUP BY dept
)
SELECT TOP 1 dept, avg_salary
FROM dept_avgs
ORDER BY avg_salary DESC;
```

Output / Result Set:

dept	avg_salary
IT	82333

Explanation: Both methods return the same result. CTEs are far more readable, especially when the same subquery would be repeated. They also help with debugging — test the CTE independently first.

★ **Key Point:** Always prefer CTE over nested subqueries for queries that need the same subquery result more than once.

Q9.8

CTE with DELETE – Remove Duplicate Records

Scenario: Remove duplicate employees keeping only the one with the lowest emp_id.

Table: employees

emp_id	name	dept	salary
1	Alice	IT	75000
2	Bob	HR	45000
3	Alice	IT	75000
4	Charlie	IT	90000
5	Bob	HR	45000

SQL Query:

```
WITH duplicates_to_delete AS (
  SELECT
    emp_id,
    ROW_NUMBER() OVER (
      PARTITION BY name, dept
      ORDER BY emp_id
    ) AS rn
  FROM employees
)
DELETE FROM duplicates_to_delete
WHERE rn > 1;

-- After deletion:
SELECT * FROM employees ORDER BY emp_id;
```

Output / Result Set:

emp_id	name	dept	salary
1	Alice	IT	75000
2	Bob	HR	45000
4	Charlie	IT	90000

Explanation: ROW_NUMBER() assigns 1 to the first occurrence (lowest emp_id) and 2, 3... to duplicates. The DELETE removes all rows where rn > 1. SQL Server supports DELETE from CTEs directly.

★ **Key Point:** This is the safest and most elegant way to delete duplicates. Always verify with SELECT first before running DELETE.

Q9.9

CTE for Pagination (OFFSET-FETCH)

Scenario: Implement pagination — show page 2 of results (rows 6 to 10) ordered by salary.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
DECLARE @page_size INT = 3;
DECLARE @page_num INT = 2; -- Page 2

WITH paginated AS (
  SELECT
    emp_id,
    name,
    dept,
    salary,
    ROW_NUMBER() OVER (
      ORDER BY salary DESC
    ) AS row_num
  FROM employees
)
SELECT
  emp_id, name, dept, salary, row_num
FROM paginated
WHERE row_num BETWEEN
  (@page_num - 1) * @page_size + 1
```

```
AND @page_num * @page_size
ORDER BY row_num;
```

Output / Result Set:

emp_id	name	dept	salary	row_num
4	Diana	Finance	60000	4
7	George	Finance	55000	5
8	Hannah	Marketing	48000	6

Explanation: ROW_NUMBER() gives sequential row numbers. The WHERE clause calculates the start and end row for any given page. Changing @page_num returns different pages.

★ **Key Point: Modern SQL: ORDER BY salary DESC OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY (SQL Server 2012+, PostgreSQL) is simpler for basic pagination.**

Q9.10

CTE for Month-over-Month Growth

Scenario: Calculate the month-over-month revenue growth percentage.

Table: monthly_sales

month	revenue
2024-01	130000
2024-02	107000
2024-03	155000
2024-04	190000
2024-05	175000

SQL Query:

```
WITH sales_with_prev AS (
  SELECT
    month,
    revenue,
    LAG(revenue, 1) OVER (
      ORDER BY month
    ) AS prev_revenue
  FROM monthly_sales
)
SELECT
  month,
  revenue,
  ISNULL(prev_revenue, 0) AS prev_month_revenue,
  CASE
```



```

WHEN prev_revenue IS NULL THEN 'N/A'
ELSE CAST(
ROUND(100.0 *
(revenue - prev_revenue)
/ prev_revenue, 2)
AS VARCHAR) + '%'
END AS mom_growth
FROM sales_with_prev
ORDER BY month;

```

Output / Result Set:

month	revenue	prev_month_revenue	mom_growth
2024-01	130000	0	N/A
2024-02	107000	130000	-17.69%
2024-03	155000	107000	44.86%
2024-04	190000	155000	22.58%
2024-05	175000	190000	-7.89%

Explanation: CTE uses LAG() to bring the previous month's revenue. The outer query calculates the percentage change. CASE handles the first month which has no previous month.

★ **Key Point:** MoM growth = (current - previous) / previous * 100. This pattern is used in every business intelligence dashboard.

CHAPTER 10

DDL, DML & Transactions

CREATE, ALTER, DROP, INSERT, UPDATE, DELETE, TRUNCATE, indexes, views, and transactions.

Q10.1

CREATE TABLE with All Major Constraints

Scenario: Create an employees table with proper data types and constraints from scratch.

Table: N/A – DDL

Concept
This is a CREATE statement

SQL Query:

```
CREATE TABLE employees (  
  emp_id INT NOT NULL  
  PRIMARY KEY  
  IDENTITY(1,1),  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  dept_id INT NULL,  
  salary DECIMAL(10,2) NOT NULL  
  DEFAULT 0,  
  hire_date DATE NOT NULL  
  DEFAULT GETDATE(),  
  is_active BIT NOT NULL DEFAULT 1,  
  manager_id INT NULL,  
  CONSTRAINT fk_dept  
  FOREIGN KEY (dept_id)  
  REFERENCES departments(dept_id),  
  CONSTRAINT fk_manager  
  FOREIGN KEY (manager_id)  
  REFERENCES employees(emp_id),  
  CONSTRAINT chk_salary  
  CHECK (salary >= 0)  
);
```

Output / Result Set:

Constraint	Column	Purpose
PRIMARY KEY	emp_id	Unique identifier, no NULLS, auto-increment
NOT NULL	name	Cannot be empty
UNIQUE	email	Each email must be distinct
DEFAULT	salary, hire_date	Fallback value if not provided
FOREIGN KEY	dept_id	Must match departments.dept_id
FOREIGN KEY	manager_id	Self-referencing — must be an existing emp_id
CHECK	salary	Cannot be negative

Explanation: PRIMARY KEY = NOT NULL + UNIQUE. IDENTITY(1,1) auto-increments starting at 1. FOREIGN KEY enforces referential integrity. CHECK validates business rules at the database level.

★ **Key Point:** Always define constraints at creation time. Adding them later to a large table is slow and error-prone.

Q10.2

ALTER TABLE – Add, Modify, and Drop Columns

Scenario: After deployment, the business requires adding a phone column, changing salary precision, and removing a deprecated column.

Table: employees (before)

emp_id	name	dept	salary	old_code
1	Alice	IT	75000	A001
2	Bob	HR	45000	B002

SQL Query:

```
-- 1. Add a new column
ALTER TABLE employees
ADD phone VARCHAR(15) NULL;

-- 2. Modify column: increase salary precision
ALTER TABLE employees
ALTER COLUMN salary DECIMAL(12, 2) NOT NULL;

-- 3. Add a column with constraint
ALTER TABLE employees
ADD created_at DATETIME DEFAULT GETDATE();

-- 4. Drop a deprecated column
ALTER TABLE employees
DROP COLUMN old_code;

-- 5. Rename a column (SQL Server)
```

```
EXEC sp_rename
'employees.name',
'full_name',
'COLUMN';
```

Output / Result Set:

Operation	Syntax	Notes
Add column	ALTER TABLE t ADD col TYPE	New column added, NULL for existing rows unless DEFAULT given
Modify column	ALTER TABLE t ALTER COLUMN col new_type	Cannot reduce size if data exists
Add with default	ADD col TYPE DEFAULT value	Existing rows get default value
Drop column	ALTER TABLE t DROP COLUMN col	Permanent — cannot undo without backup
Rename column	sp_rename (SQL Server)	PostgreSQL: ALTER TABLE t RENAME COLUMN old TO new

Explanation: ALTER TABLE is for modifying an existing table structure. Each ALTER is a separate statement. Dropping a column with data is irreversible — always backup first.

★ **Key Point:** In production: test ALTER statements on a copy first. Modifying a large table can lock it and cause downtime.

Q10.3

INSERT – Single Row, Multiple Rows, and INSERT SELECT

Scenario: Insert employee records into the table — including bulk insert from another table.

Table: new_employees

name	dept	salary	hire_date
Priya	IT	72000	2024-01-15
Rahul	HR	44000	2024-02-01

SQL Query:

```
-- 1. Single row insert (explicit columns)
INSERT INTO employees
(name, dept, salary, hire_date)
VALUES
('Priya', 'IT', 72000, '2024-01-15');

-- 2. Multiple rows in one statement
INSERT INTO employees
(name, dept, salary, hire_date)
VALUES
```

```

('Rahul', 'HR', 44000, '2024-02-01'),
('Sana', 'Finance', 58000, '2024-02-15'),
('Arjun', 'IT', 81000, '2024-03-01');

-- 3. INSERT...SELECT (copy from another table)
INSERT INTO employees
(name, dept, salary, hire_date)
SELECT
name, dept, salary, hire_date
FROM new_employees
WHERE hire_date >= '2024-01-01';

```

Output / Result Set:

Method	Use Case	Notes
Single INSERT	Add one record	Always specify column names — avoids breaks if columns are added
Multi-row INSERT	Bulk insert few rows	Faster than individual INSERTs, single transaction
INSERT SELECT	Copy from another table/query	No VALUES clause; column count must match

Explanation: Always specify column names in INSERT — positional inserts (without column list) break silently if columns are reordered or added.

★ **Key Point:** INSERT without column names is dangerous. If anyone adds a column to the table, your query fails or inserts wrong data.

Q10.4

UPDATE – Single Column, Multiple Columns, and UPDATE with JOIN

Scenario: Update employee salaries: give a 10% raise to IT department employees.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
-- 1. Simple UPDATE
UPDATE employees
SET salary = salary * 1.10
WHERE dept = 'IT';

-- 2. Update multiple columns
UPDATE employees
SET
salary = salary * 1.10,
hire_date = GETDATE()
WHERE emp_id = 1;

-- 3. UPDATE with JOIN (SQL Server)
UPDATE e
SET e.salary = e.salary * d.raise_pct
FROM employees e
INNER JOIN dept_raises d
ON e.dept = d.dept
WHERE d.year = 2024;
```

Output / Result Set:

emp_id	name	dept	old_salary	new_salary
1	Alice	IT	75000	82500
3	Charlie	IT	90000	99000
6	Fatima	IT	82000	90200

Explanation: UPDATE with JOIN allows updating rows based on data in another table — very powerful for bulk conditional updates. Always include WHERE to avoid updating all rows.

★ **Key Point:** Before any UPDATE, run a SELECT with the same WHERE clause to see exactly which rows will change.

Q10.5

DELETE vs TRUNCATE vs DROP – Know the Difference

Scenario: Understand when to use DELETE, TRUNCATE, and DROP — a very common interview topic.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000

emp_id	name	dept	city	salary
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
-- 1. DELETE: Remove specific rows (logged, can rollback)
DELETE FROM employees
WHERE dept = 'HR';

-- DELETE all rows (slow, logged)
DELETE FROM employees;

-- 2. TRUNCATE: Remove all rows (fast, minimal logging)
TRUNCATE TABLE employees;

-- 3. DROP: Remove the entire table structure
DROP TABLE employees;
```

Output / Result Set:

Operation	Removes	WHERE clause	Rollback	Resets IDENTITY	Speed
DELETE	Specific rows	Yes	Yes (logged)	No	Slow (row-by-row)
TRUNCATE	All rows	No	Limited	Yes	Fast (page-level)
DROP	Table + structure	No	Yes (in txn)	N/A	Instant

Explanation: DELETE removes rows one at a time and logs each row — rollback-able. TRUNCATE removes all data by deallocating data pages — much faster. DROP removes the entire table definition.

★ **Key Point:** Interview answer: **TRUNCATE is faster than DELETE for removing all rows because it doesn't log individual row deletions. TRUNCATE cannot be used if the table has foreign key references.**

Q10.6

Soft Delete – Archive Instead of Permanently Deleting

Scenario: Instead of permanently deleting employees who resign, mark them as inactive.

Table: employees

emp_id	name	dept	salary	is_active	resigned_at
1	Alice	IT	75000	1	NULL
2	Bob	HR	45000	1	NULL
3	Charlie	IT	90000	1	NULL

emp_id	name	dept	salary	is_active	resigned_at
4	Diana	Finance	60000	1	NULL

SQL Query:

```
-- Soft delete: mark as inactive instead of DELETE
UPDATE employees
SET
  is_active = 0,
  resigned_at = GETDATE()
WHERE emp_id = 2;

-- Query active employees only
SELECT
  emp_id,
  name,
  dept,
  salary,
  is_active
FROM employees
WHERE is_active = 1
ORDER BY emp_id;
```

Output / Result Set:

emp_id	name	dept	salary	is_active
1	Alice	IT	75000	1
3	Charlie	IT	90000	1
4	Diana	Finance	60000	1

Explanation: Soft delete preserves historical data for auditing and reporting. A flag column (is_active, is_deleted) marks records as inactive. All queries filter by the flag.

★ **Key Point:** Most production systems use soft delete. Add a created_at and updated_at timestamp to every table for full audit trail.

Q10.7

Upsert – INSERT if Not Exists, UPDATE if Exists

Scenario: Update an employee's salary if they exist; insert a new record if they don't.

Table: employees

emp_id	name	dept	city	salary
1	Alice	IT	Bangalore	75000
2	Bob	HR	Mumbai	45000
3	Charlie	IT	Hyderabad	90000
4	Diana	Finance	Delhi	60000

emp_id	name	dept	city	salary
5	Ethan	HR	Bangalore	45000
6	Fatima	IT	Chennai	82000
7	George	Finance	Mumbai	55000
8	Hannah	Marketing	Delhi	48000

SQL Query:

```
-- SQL Server: MERGE statement (Upsert)
MERGE employees AS target
USING (
VALUES
(1, 'Alice', 'IT', 80000),
(9, 'NewPerson', 'IT', 65000)
) AS source (emp_id, name, dept, salary)
ON target.emp_id = source.emp_id
WHEN MATCHED THEN
UPDATE SET
salary = source.salary
WHEN NOT MATCHED THEN
INSERT (emp_id, name, dept, salary)
VALUES (source.emp_id, source.name,
source.dept, source.salary);
```

Output / Result Set:

emp_id	name	dept	salary	action
1	Alice	IT	80000	UPDATED (was 75000)
9	NewPerson	IT	65000	INSERTED (new record)

Explanation: MERGE combines INSERT and UPDATE in one atomic operation. WHEN MATCHED updates existing records; WHEN NOT MATCHED inserts new ones. Eliminates race conditions in concurrent updates.

★ **Key Point:** MySQL equivalent: **INSERT ... ON DUPLICATE KEY UPDATE**. PostgreSQL: **INSERT ... ON CONFLICT DO UPDATE (UPSERT)**. These are different syntax but same concept.

Q10.8

CREATE INDEX – Speed Up Query Performance

Scenario: Queries filtering by dept are slow on a large table. Add an index to speed them up.

Table: employees (1M rows)

Operation	Without Index	With Index
SELECT WHERE dept=	Full Table Scan	Index Seek
Rows scanned	1,000,000	~50,000
Response time	~2 seconds	~0.05 seconds

SQL Query:

```
-- Create a simple non-clustered index
CREATE INDEX idx_emp_dept
ON employees (dept);

-- Create a composite index
CREATE INDEX idx_emp_dept_salary
ON employees (dept, salary);

-- Create a unique index
CREATE UNIQUE INDEX idx_emp_email
ON employees (email);

-- View existing indexes
SELECT
name AS index_name,
type_desc AS index_type
FROM sys.indexes
WHERE object_id = OBJECT_ID('employees');

-- Drop an index
DROP INDEX idx_emp_dept
ON employees;
```

Output / Result Set:

Index Type	Best Used For	Trade-off
Clustered Index	Primary Key lookups	Only 1 per table; defines physical row order
Non-Clustered	WHERE, JOIN, ORDER BY cols	Multiple allowed; extra storage for index
Composite Index	Multi-column WHERE clauses	Column order matters — left-most column first
Unique Index	Enforce uniqueness + speed	Prevents duplicates automatically

Explanation: Indexes are like a book's index — they allow SQL Server to find rows without scanning every row. They dramatically speed up SELECT but slightly slow down INSERT/UPDATE/DELETE.

★ **Key Point:** Index the columns you use most in WHERE, JOIN ON, and ORDER BY. Don't over-index — every index consumes storage and slows writes.

Q10.9

Transactions – COMMIT and ROLLBACK

Scenario: Transfer █10,000 between two accounts atomically — both operations must succeed or both must fail.

Table: accounts

account_id	name	balance
1	Alice	50000
2	Bob	20000

SQL Query:

```
BEGIN TRANSACTION;

-- Step 1: Deduct from Alice
UPDATE accounts
SET balance = balance - 10000
WHERE account_id = 1;

-- Check if Alice has enough balance
IF (SELECT balance FROM accounts
WHERE account_id = 1) < 0
BEGIN
    ROLLBACK TRANSACTION;
    PRINT 'Insufficient funds. Rolled back.';
END
ELSE
BEGIN
    -- Step 2: Add to Bob
    UPDATE accounts
    SET balance = balance + 10000
    WHERE account_id = 2;

    COMMIT TRANSACTION;
    PRINT 'Transfer successful.';
END
```

Output / Result Set:

account_id	name	before_balance	after_balance
1	Alice	50000	40000
2	Bob	20000	30000

Explanation: A transaction groups SQL statements into an atomic unit — either ALL succeed (COMMIT) or ALL fail (ROLLBACK). This guarantees data consistency (ACID properties).

★ **Key Point: ACID: Atomicity (all or nothing), Consistency (valid state), Isolation (concurrent safe), Durability (committed = permanent). Transactions are the foundation of reliable databases.**

Q10.10

Views – Create Reusable Query Definitions

Scenario: Create a view for the HR dashboard that shows active employees with their department and tenure.

Table: employees

emp_id	name	dept	salary	hire_date	dob
1	Alice	IT	75000	2022-03-15	1995-06-10
2	Bob	HR	45000	2021-07-01	1990-11-25
3	Charlie	IT	90000	2020-01-10	1988-03-18
4	Diana	Finance	60000	2023-05-20	1993-08-07
5	Ethan	HR	42000	2022-11-30	1997-01-14
6	Fatima	IT	82000	2021-04-18	1994-09-30
7	George	Finance	55000	2019-08-25	1985-12-05

SQL Query:

```
-- Create a view
CREATE VIEW vw_active_employees AS
SELECT
  e.emp_id,
  e.name,
  e.dept,
  e.salary,
  e.hire_date,
  DATEDIFF(YEAR, e.hire_date, GETDATE())
  AS years_of_service,
  CASE
    WHEN salary < 45000 THEN 'Junior'
    WHEN salary < 70000 THEN 'Mid-Level'
    WHEN salary < 85000 THEN 'Senior'
    ELSE 'Lead'
  END AS salary_band
FROM employees
WHERE is_active = 1;

-- Use the view like a table
SELECT * FROM vw_active_employees
WHERE dept = 'IT'
ORDER BY salary DESC;
```

Output / Result Set:

emp_id	name	dept	salary	years_of_ser vice	salary_band
3	Charlie	IT	90000	3	Lead
6	Fatima	IT	82000	2	Senior
1	Alice	IT	75000	1	Senior

Explanation: A VIEW is a stored query that acts like a virtual table. It doesn't store data — it runs the underlying query each time. Views simplify complex queries, enforce security, and abstract schema changes.

★ **Key Point:** Use views for frequently used complex queries. Note: views with complex logic can be slow — consider indexed views or materialized views for performance-critical cases.